

Optimizing Open-Source LLMs for CRM Automation in E-Commerce

by

Ajitha Sivakulasingam

CS/2019/012

Under the supervision of

Dr.(Mrs). M. Carmel Wijegunasekara

CSCI 43018 – Research Project

Thesis

Academic year

2022/2023

Submitted in partial fulfilment of the requirements for the
Bachelor of Science Honours in Computer Science Degree



Faculty of Computing and Technology

University of Kelaniya

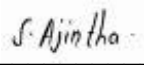
20/08/2025

Declaration

I, Ajintha Sivakulasingam, hereby declare that the thesis entitled “Optimizing Open-Source LLMs for CRM Automation in E-Commerce”, submitted to the Faculty of Computing and Technology, University of Kelaniya, in fulfilment of the requirements for the award of the degree of Bachelor of Science Honours in Computer Science, is the result of my own independent work carried out under the guidance and supervision of Dr.(Mrs). M. Carmel Wijegunasekara. This work has not been submitted, in whole or in part, to any conference, journal, book, university, or other institution for the purpose of obtaining any degree, diploma, or other academic qualification. I confirm that all sources of information and ideas from other works have been properly acknowledged in accordance with accepted academic conventions.

Name of the Student: Ajintha S.

Student Number: CS/2019/012

Signature of the Student: 

Date: 20/08/2025

Name of the Supervisor: Dr.(Mrs). M. Carmel Wijegunasekara

Department: _____

Signature of the Supervisor: _____

Date: _____

Abstract

The increasing scale of customer interactions in the e-commerce sector has created a critical demand for automated, intelligent Customer Relationship Management (CRM) systems. While proprietary Large Language Models (LLMs) such as GPT-4 perform well in such tasks, their high costs make them inaccessible to many small and medium-sized enterprises (SMEs). Open-source alternatives like LLaMA-3.1 offer a more affordable solution, but face limitations in accuracy and contextual reasoning when applied to complex CRM tasks. This study investigates whether these limitations can be overcome using a multi-agent system.

The aim of the research is to optimize open-source LLMs for CRM automation in e-commerce by using task-specific agents to improve performance in three key tasks: Named Entity Disambiguation (NED), Policy Violation Identification (PVI), and Knowledge-based Question Answering (KQA). The primary objective was to design and evaluate a scalable framework that enhances the accuracy and reliability of open-source LLMs in automating these three critical CRM tasks. The CRMArena benchmark, developed by Salesforce AI Research, was used as the test environment, simulating realistic CRM scenarios across 130 queries per task.

Two model variants, LLaMA-3.1 70B and 405B, were tested using LangGraph-based multi-agent workflows. The 70B model achieved 92.31% accuracy in NED, 59.23% in PVI, and 85.38% in KQA, while the 405B model reached 86.92% (NED), 36.15% (PVI), and 84.62% (KQA). These results outperformed the CRMArena benchmark scores for open-source models typically under 55%, showing that multi-agent task decomposition significantly enhances model effectiveness. The solution was also cost-efficient, with per-task inference costs as low as \$0.1

The key conclusion of this study is that a well-structured multi-agent system can substantially improve the performance of open-source LLMs in CRM scenarios, enabling cost-effective and accurate automation comparable to that of proprietary solutions. Moreover, the architecture proved modular, interpretable, and adaptable, allowing each agent to specialize in specific subtasks and improving overall reasoning and reliability.

As a recommendation for future work, this framework could be further enhanced through domain-specific fine-tuning, integration of memory and feedback loops, and deployment in real-world CRM environments to validate its robustness under live customer interaction data.

Acknowledgement

I would like to express my heartfelt gratitude to everyone who supported and guided me throughout the completion of my thesis titled “Optimizing Open-Source LLMs for CRM Automation in E-commerce.”

First and foremost, I extend my deepest appreciation to my supervisor, Dr. (Mrs.) M. Carmel Wijegunasekara, for her invaluable guidance, constant encouragement, and insightful feedback throughout the research journey. Her expertise and support were instrumental in shaping this work and helping me grow both technically and academically.

I also wish to thank the academic staff and lecturers of the Faculty of Computing and Technology, University of Kelaniya, whose teaching and resources provided a strong foundation for this research. I am especially grateful for the supportive environment and facilities made available during this journey.

My sincere thanks go to my fellow students and friends for their encouragement, thoughtful discussions, and moral support. Their companionship made this journey more enjoyable and less overwhelming.

Lastly, I am deeply thankful to my family for their unconditional love, patience, and belief in me. Their continuous support has been a pillar of strength throughout my academic life.

Thank you all for being a part of this meaningful and enriching journey.

Table of Contents

Declaration.....	2
Abstract.....	3
Acknowledgement	4
Table of Contents.....	5
Table of Figures	8
Tables.....	9
Abbreviations.....	9
Chapter 1	10
Introduction.....	10
1.1 Introduction	10
1.2 Research Problem.....	10
1.3 Aims and Objectives	11
1.4 Motivation	12
1.5 Scope and Significance	12
1.6 The Layout of the Thesis.....	12
Chapter 2	14
Literature Review.....	14
2.1 Introduction	14
2.2 The Importance of CRM Automation	15
2.3 LLMs in CRM Automation.....	16
2.3.1 LLMs in Automating CRM Tasks.....	16
2.3.2 Challenges and Limitations of LLMs in CRM.....	17
2.3.3 Gaps in Open-Source LLMs.....	17
2.4 Multi-Agent Frameworks in CRM.....	18
2.5 Conclusion.....	19

Chapter 3	21
Methodology	21
3.1 Introduction	21
3.2 Data Collection.....	21
3.2.1 Data Description	22
3.2.2 Data Collection Method.....	23
3.3 Data Analysis	24
3.3.1 Data Preparation	24
3.4 Design.....	27
3.4.1 Objective of the Design	27
3.4.2 System Architecture	28
3.4.3 Agents and Sub-Agents	29
3.5 Implementation.....	33
3.5.1 Environment Setup	33
3.5.2 Code Structure	34
3.5.3 LLaMA-3.1 Integration	36
3.5.4 Agent Connection via LangGraph.....	37
3.5.5 Integration Challenges.....	37
3.6 Evaluation.....	38
3.6.1 Purpose of Evaluation.....	38
3.6.2 Test Dataset and Setup	38
3.6.3 Evaluation Metrics.....	39
Chapter 4	41
Results.....	41
4.1 Introduction	41
4.2 Accuracy and Inference Cost	41

4.3 BERTScore Results for KQA	42
4.4 Example Predictions.....	43
Chapter 5	44
Discussion	44
5.1 Introduction	44
5.2 Interpretation of Performance Across Tasks	44
5.3 Inference Cost Implications	45
5.4 BERTScore Findings for KQA Task	45
5.5 Comparison with Prior Studies	46
5.6 Research Limitations.....	47
Chapter 6	49
Conclusion	49
6.1 Alignment with Project Aims and Objectives.....	49
6.2 Summary of Key Results.....	49
6.3 Conclusions Drawn from Results.....	50
6.4 Future Directions.....	50
Chapter 7	51
Recommendations.....	51
7.1 Enhancing PVI Performance	51
7.2 Optimizing Inference Cost Scalability	51
7.3 Expanding Model and Task Diversity.....	52
7.4 Practical Deployment for SMEs.....	52
References.....	53
Appendices.....	55
Appendix A	55
CRMArena Schema (json format).....	55

Appendix B	59
Appendix C	60
Implementation Details	60
C.1 Prompts	60
C.2 SOQL / SOSL Queries	63
C.3 Prototype UI screenshots	64

Table of Figures

Figure 1: Methodology Overview Diagram.....	21
Figure 2: CRMArena Logo.....	21
Figure 3: Salesforce client connection code	23
Figure 4: Retrieve AccountId from Contact	26
Figure 5: Retrieve ProductID and Name from OrderItem	27
Figure 6: Task Classifier Agent Workflow Diagram.....	29
Figure 7: NED Agent Workflow Diagram.....	30
Figure 8: PVI Agent Workflow Diagram	31
Figure 9: KQA Agent Workflow Diagram	32
Figure 10: Prototype Flowchart Diagram	35
Figure 11: Accuracy formula for NED and PVI.....	40
Figure 12: BERTScore formula for KQA.....	40
Figure 13: CRMArena Sandbox Schema Diagram.....	59
Figure 14: Home Page of Prototype.....	64
Figure 15: NED Task UI.....	65
Figure 16:NED Output Page.....	65
Figure 17: KQA Output Page	66
Figure 18: PVI Task UI.....	66

Figure 19: PVI Output Page.....	67
---------------------------------	----

Tables

Table 1: Comparison of CRM Automation Approaches	19
Table 2: The number of entries per object.	22
Table 3: Accuracy and Inference Cost for NED, PVI, and KQA	41
Table 4: BERTScore F1 Results for KQA Queries	42
Table 5: Sample Outputs for NED, PVI, and KQA	43
Table 6: Comparison of Proposed System with Prior Studies	46

Abbreviations

- CRM - Customer Relationship Management
- LLM - Large Language Model
- KQA - Knowledge Question Answering
- NED - Named Entity Disambiguation
- PVI - Policy Violation Identification
- MAS - Multi-Agent System
- SME - Small and Medium-sized Enterprises
- SOQL - Salesforce Object Query Language
- SOSL - Salesforce Object Search Language

Chapter 1

Introduction

1.1 Introduction

The e-commerce sector has experienced rapid growth in recent years, driven by increased consumer reliance on online shopping platforms. According to Statista [1], global retail e-commerce sales have shown a consistent upward trend, highlighting the need for more robust digital infrastructure. This surge has intensified the demand for effective Customer Relationship Management (CRM) systems capable of managing large volumes of customer interactions, delivering personalized experiences, and maintaining adherence to business policies. Traditional CRM systems, often dependent on manual processes, struggle to keep up with this expansion, leading to inefficiencies, higher costs, and lower customer satisfaction [2]. Artificial Intelligence (AI), particularly Large Language Models (LLMs), offers great potential to automate CRM tasks, streamlining operations, improving personalization, and enabling data-driven decisions [2].

However, open-source LLMs, such as LLaMA-3.1, face significant challenges in e-commerce CRM automation. The CRMArena benchmark [3] indicates that these models achieve suboptimal performance, with success rates below 55% in critical tasks like Knowledge Question Answering (KQA), Named Entity Disambiguation (NED), and Policy Violation Identification (PVI). These limitations arise from difficulties in understanding nuanced contexts, executing multi-step tasks, and integrating with CRM system interfaces. In contrast, proprietary models offer higher accuracy but come with prohibitive costs, making them inaccessible for many e-commerce businesses, especially startups and small-to-medium enterprises (SMEs). This performance gap highlights a critical need for cost-effective, reliable open-source LLM solutions tailored for professional CRM tasks in e-commerce [3].

1.2 Research Problem

In the e-commerce industry, Customer Relationship Management (CRM) systems are vital for managing customer interactions, resolving order issues, and answering product-related questions. Large Language Models (LLMs) show promise in automating these tasks, but open-source LLMs, such as LLaMA-3.1, face significant challenges in achieving the required

accuracy, contextual understanding, and policy alignment. Specifically, these models struggle with:

- **Named Entity Disambiguation (NED):** Correctly identifying and distinguishing product names or terms in customer interactions.
- **Policy Violation Identification (PVI):** Detecting whether customer actions breach company rules.
- **Knowledge Question Answering (KQA):** Providing accurate, policy-compliant responses to product or policy-related queries.

The CRMArena benchmark [3], reveals that open-source LLMs perform poorly compared to advanced proprietary models, with success rates below 55% in these task. This performance gap is evident in frameworks like Act, ReAct, and Function Calling, which structure LLM task execution but fail to deliver consistent reliability. The lack of domain-specific optimization further limits open-source LLMs' suitability for professional e-commerce CRM environments, where precision and compliance are critical.

1.3 Aims and Objectives

The aim of this research is to develop and evaluate a multi-agent framework that optimizes open-source LLMs for e-commerce CRM automation, enhancing their accuracy and reliability to compete with proprietary models. The specific objectives are:

1. Investigate the performance limitations of open-source LLMs in CRM tasks, focusing on KQA, NED, and PVI, through a comprehensive literature review.
2. Design a multi-agent framework using LLaMA-3.1 and LangGraph, incorporating task-specific agents and workflow decomposition to address contextual and multi-step challenges.
3. Implement the framework in a simulated e-commerce environment, testing its performance on 130 diverse CRM task queries to achieve a target accuracy exceeding 55%.
4. Compare the performance of optimized models against existing open-source and proprietary models using the CRMArena benchmark.

5. Recommend strategies for further optimization, including domain-specific fine-tuning and real-world testing, to support broader adoption of open-source LLMs in e-commerce CRM.

1.4 Motivation

The motivation for this research is both practical and academic. Practically, the high costs of proprietary LLMs restrict advanced CRM automation to large enterprises, leaving SMEs dependent on less efficient manual or semi-automated systems. A cost-effective open-source solution could level the playing field, enabling smaller businesses to deliver personalized, efficient customer experiences and compete in the dynamic e-commerce market. To address this gap, this study proposes a Multi-Agent System leveraging open-source LLMs for CRM automation.

Academically, this study advances the field of AI-driven CRM by exploring the untapped potential of multi-agent frameworks in optimizing open-source LLMs. By innovating task decomposition and agent collaboration, the research contributes to the broader understanding of scalable AI solutions for automation challenges.

1.5 Scope and Significance

The scope of this thesis covers the design, implementation, and evaluation of a multi-agent framework for open-source LLMs in e-commerce CRM automation, focusing on KQA, NED, and PVI tasks. The study uses simulated CRM tasks and synthetic datasets to ensure ethical data handling, with recommendations provided for real-world deployment. The significance lies in its potential to narrow the performance gap between open-source and proprietary LLMs, offering a cost-effective solution for e-commerce businesses and enriching academic insights into multi-agent systems for AI-driven automation.

1.6 The Layout of the Thesis

This thesis is structured to systematically present the development, evaluation, and implications of a multi-agent system (MAS) designed to automate Customer Relationship Management (CRM) tasks—Named Entity Disambiguation (NED), Policy Violation Identification (PVI), and Knowledge-based Question Answering (KQA). The document is

organized into seven chapters, each addressing a specific aspect of the research. The following outlines the structure and purpose of each chapter:

1. **Introduction** introduces the research problem, outlining the significance of automating CRM tasks in eCommerce, particularly for small-to-medium enterprises (SMEs). It defines the project's aims and objectives, focusing on achieving high accuracy, cost-effectiveness, and semantic reliability with the MAS, and presents the scope and context of the study.
2. **Literature Review** surveys existing studies on CRM automation and agent-based systems, including benchmarks like CRMArena [3], WorkArena [8], WebArena [9], AssistantBench [10], and Thakkar & Yadav [14]. It identifies gaps in single-agent systems and highlights the need for multi-agent approaches, positioning the current research within the field.
3. **Methodology** describes the research design, including the CRMArena dataset's structure, the MAS architecture, and evaluation methods. It details data collection, analysis, and testing procedures conducted in Google Colab.
4. **Results** presents the MAS's performance results, including accuracy, inference costs, and BERTScore F1 results for KQA, along with example predictions.
5. **Discussion** interprets the results, comparing the MAS's performance with prior studies, analyzing strengths, limitations, and implications for eCommerce CRM automation.
6. **Conclusion** summarizes the key findings, draws conclusions about the MAS's effectiveness, and reflects on how these align with the project's aims of delivering accurate, cost-effective, and reliable CRM automation for SMEs.
7. **Recommendation** proposes actionable steps to enhance the MAS, including improving PVI performance, optimizing inference costs, expanding model and task diversity, and facilitating SME deployment, based on the project's findings and limitations.

This structure ensures a logical progression from problem identification to evaluation, interpretation, and practical recommendations, providing a comprehensive examination of the MAS's potential for eCommerce CRM automation.

Chapter 2

Literature Review

2.1 Introduction

The aim of this literature review is to take a deep dive into the current research surrounding the use of artificial intelligence (AI), particularly focusing on open-source large language models (LLMs) like LLaMA-3.1 70B and 405B, in automating customer relationship management (CRM) within the e-commerce sector. By bringing together various studies on AI-powered CRM systems, we'll look at how open-source LLMs perform in tasks such as Named Entity Disambiguation (NED), Policy Violation Identification (PVI), and Knowledge Question Answering (KQA). This review seeks to pinpoint advancements, limitations, and gaps in research. It will lay the groundwork for creating a multi-agent framework aimed at optimizing open-source LLMs, boosting their accuracy and reliability for e-commerce CRM automation, and providing a cost-effective alternative to proprietary models.

The scope of this literature review encompasses four key areas critical to the research objectives:

- **AI-Driven CRM Systems:** Exploring how AI, including chatbots and predictive analytics, enhances customer interactions, operational efficiency, and data-driven decision-making in e-commerce CRM, as demonstrated by platforms like Salesforce [4], [5].
- **Open-Source LLMs in CRM:** Analyzing the capabilities and limitations of open-source LLMs, particularly LLaMA-3.1 models, in executing complex CRM tasks, with a focus on benchmarks like CRMArena that highlight performance gaps [3].
- **Multi-Agent Frameworks for LLM Optimization:** Investigating the potential of multi-agent frameworks, such as those using LangGraph and CrewAI, to improve open-source LLM performance in CRM tasks by enabling task specialization and collaboration [6], [7].

2.2 The Importance of CRM Automation

According to Salesforce (n.d.) [4], using automation in CRM systems allows organizations to "save time, reduce errors, and improve productivity" by streamlining tasks like lead scoring, responding to emails, and assigning tasks. By cutting down on human involvement in everyday operations, companies can use their resources more wisely, which boosts overall efficiency.

Automation is transforming customer experiences by diving deep into behavioral data and anticipating what customers really want. As highlighted in a Medium article from 2018 [5], incorporating AI into CRM systems empowers businesses to "offer personalized recommendations, foresee customer challenges, and deliver support in real-time." With AI-powered CRM tools, companies can categorize customers based on their preferences, buying habits, and engagement trends, making sure that marketing and sales strategies hit the mark every time.

Automated CRM systems gather and analyze a wealth of customer data, offering valuable insights for businesses. According to Salesforce (n.d.) [4], automation plays a key role in "tracking customer interactions, forecasting sales trends, and identifying opportunities for upselling." This data-driven strategy helps companies make well-informed decisions, ultimately boosting customer satisfaction and retention rates.

As businesses grow, relying on manual CRM processes just doesn't cut it anymore. Automation steps in to provide the scalability needed to manage a surge in customer interactions without a corresponding spike in staffing costs. An article from Medium (2018) [5] points out that AI-driven CRM systems "adapt to growing business needs, ensuring seamless customer experiences even as operations scale." This ability to scale is especially vital for startups and larger companies that are looking to expand quickly.

In summary, CRM automation is essential in today's business landscape, as it boosts efficiency, enhances personalization, and supports data-driven strategies. Insights from Salesforce and Medium highlight how AI and automation are revolutionizing traditional CRM into a smart, adaptable system that strengthens customer relationships and propels business success. Future studies should delve into the long-term effects of AI integration in CRM and what it means for customer loyalty and gaining a competitive edge.

2.3 LLMs in CRM Automation

Large language models (LLMs) have become essential tools for streamlining customer relationship management (CRM) tasks in the e-commerce world, promising to boost both operational efficiency and customer satisfaction. Yet, when it comes to implementing these models—especially open-source ones like LLaMA-3.1—there are notable hurdles to overcome, primarily due to their struggles with accuracy and contextual understanding in the messy reality of real-world situations [3].

2.3.1 LLMs in Automating CRM Tasks

The CRMArena benchmark [3] shows that LLM-based agents are set to transform CRM by taking care of everyday tasks like answering customer questions, spotting policy violations, and sorting out order problems. This not only boosts efficiency but also enhances customer satisfaction. For example, LLMs can understand customer inquiries, pull out key information, and deliver precise answers by tapping into external data sources like product catalogs or policy databases [3].

The ReAct framework brings together reasoning and action, boosting the capabilities of LLMs to tackle various tasks by allowing for step-by-step planning and interaction with APIs. This was clearly shown in settings like WebShop, where it surpassed traditional methods [8]. Likewise, benchmarks such as WorkArena illustrate how LLMs can engage with enterprise software (like ServiceNow) to carry out tasks such as updating customer records or generating reports, effectively mimicking the work of knowledge workers [8].

Beyond just automating tasks, LLMs are transforming how businesses interact with customers by sifting through vast amounts of data to anticipate their needs and customize responses. In realistic online settings like WebArena [9], LLMs can seamlessly navigate e-commerce sites to help with various shopping activities, whether it's searching for products or tracking orders, all through natural language commands. AssistantBench [10] showcases how LLMs can tackle tedious tasks like drafting emails or pulling up customer details, which are essential for effective CRM.

These features are perfectly in tune with the e-commerce sector's demand for round-the-clock support and insights driven by data, as LLMs can work alongside CRM tools to create a comprehensive view of customers and offer proactive assistance. However, putting LLMs into

practice for CRM automation does come with its fair share of challenges, especially when it comes to open-source models.

2.3.2 Challenges and Limitations of LLMs in CRM

While LLMs have a lot of potential, they still struggle to deliver consistent performance in CRM tasks, mainly because real-world situations can be quite complex. The CRMarena benchmark [3] shows that even the most advanced LLM agents manage to succeed in fewer than 40% of tasks when using ReAct prompting, and only about 55% with specially designed function-calling tools. This highlights a significant reliability gap. The low success rate is largely due to challenges in grasping nuanced contexts, managing multi-step tasks, and effectively interacting with CRM system APIs. For instance, tasks like Named Entity Disamiguation (NED) require accurate entity recognition, while Policy Violation Identification (PVI) involves navigating intricate policy frameworks—both of which put a strain on LLMs' ability to reason contextually.

Other benchmarks back up these findings. WorkBench [11], which evaluates AI agents on 690 workplace tasks, reveals that even the top-performing model (GPT-4) only hits a success rate of 43%. In contrast, open-source models like Llama2-70B barely scrape by at just 3%. The errors range from using the wrong tools to making serious mistakes, like sending emails to the wrong person, which really raises concerns about reliability. Similarly, WebArena [9] indicates that GPT-4-based agents manage to complete only 14.41% of realistic web tasks, while humans achieve a much higher rate of 78.24%. This gap is largely due to the AI's difficulties with dynamic web environments and multi-step planning. AssistantBench [10] also points out that no model surpasses 26% accuracy on complex web tasks, facing challenges like hallucination and poor navigation on the web. These issues become even more pronounced in software-related tasks, as shown in SWE-bench [12], where Claude 2 only resolves 1.96% of GitHub issues, highlighting the limitations of LLMs when it comes to tackling intricate, context-sensitive problems[9].

2.3.3 Gaps in Open-Source LLMs

The performance gaps are especially noticeable when it comes to open-source LLMs, which tend to fall short compared to proprietary models in CRM automation. According to CRMarena's findings [3], open-source models like LLaMA-3.1 have a tough time hitting the accuracy levels needed for professional CRM tasks, with their success rates trailing behind proprietary options like GPT-4 or Claude. WorkArena [8] points out a significant performance

gap, highlighting that open-source models often underperform in enterprise software tasks due to their limited reasoning and tool-use capabilities. This disparity is particularly crucial for e-commerce, where achieving high accuracy in tasks like knowledge question answering (KQA) or policy violation identification (PVI) is vital for maintaining customer trust and compliance.

The absence of solid, real-world benchmarks for CRM has really held back the assessment and enhancement of open-source LLMs[3]. CRMArena [3] tries to tackle this issue by simulating realistic tasks, but the results indicate that open-source LLMs have a tough time with general-purpose tools, managing to achieve less than 40% success. On top of that, these open-source models often miss out on the domain-specific fine-tuning that proprietary systems offer, which limits their effectiveness in CRM applications [8]. Although the ReAct framework boosts performance by combining reasoning and action, its advantages are limited for open-source LLMs unless they undergo further optimization [9]. WorkBench [11] and AssistantBench [10] highlight the necessity for better task segmentation and contextual understanding, which are areas where open-source models really fall short.

2.4 Multi-Agent Frameworks in CRM

Multi-agent systems (MAS) have emerged as a promising approach to address the limitations of open-source LLMs in Customer Relationship Management (CRM) automation, as evidenced by benchmarks like CRMArena [3]. These systems leverage specialized agents to enhance task efficiency and personalization in e-commerce CRM.

Studies like Wang et al. [13] suggest that MAS enhance CRM tasks, such as Knowledge-augmented Question Answering, by distributing subtasks across specialized agents, potentially improving accuracy over single-model LLMs. For example, task decomposition allows agents to focus on specific functions, such as query analysis or data retrieval, which may improve performance in complex CRM scenarios.

Evaluations from WorkArena and WebArena [8], [9] demonstrate that MAS excel in web-based CRM tasks by assigning agents to functions like data scraping or compliance checks. These systems outperform monolithic LLMs in contextual understanding and multi-step reasoning, addressing performance gaps in open-source models.

Thakkar and Yadav [14] propose a multimodal MAS for personalized e-commerce recommendations, comprising three agents: a Product Recommendation Agent, a Multimodal

Recommendation Agent, and a Market Analysis Agent. Using LLMs like Gemini-1.5-pro and LLaMA-70B, the system integrates text, images, and real-time market trends to deliver tailored suggestions. The Product Recommendation Agent analyses user queries, the Multimodal Agent processes images for visual-based recommendations, and the Market Analysis Agent aligns suggestions with market trends. Evaluated on metrics like Precision at K, Recall at K, and ROUGE, LLaMA-70B achieved a Recall at K of 1.0 and NDCG of 1.0 for product recommendations, while Gemma2-9b-it scored a ROUGE-1 F-score of 0.5042 for market trend analysis [14]. This multimodal approach enhances recommendation accuracy compared to text-only systems.

2.5 Conclusion

The integration of Large Language Models (LLMs) into Customer Relationship Management (CRM) automation offers significant potential for e-commerce, enabling efficient handling of customer interactions and personalized experiences. However, prior research reveals substantial challenges with open-source LLMs in achieving reliable performance for professional CRM tasks. Table 1 summarizes key studies on CRM automation approaches, highlighting their tasks, features, limitations, and performance metrics.

Table 1: Comparison of CRM Automation Approaches

Study	Main Tasks	Key features	Limitations	Accuracy	System type
CRMArena [3]	Automates CRM tasks using manually crafted function calling, real-world simulation	LLMs (LLaMA-3.1, GPT-4), ReAct, Function-Calling	Low accuracy, struggles with context and multi-step tasks	<55%	Single-Agent System
WorkArena [8]	Enterprise software tasks, mimics knowledge workers	LLMs, Web-based agents	Open-source models underperform, limited reasoning	GPT-4: 43%, LLaMA2-70B: 3%	Single-Agent System

WebArena [9]	Realistic web tasks, e-commerce support	LLMs, Web navigation agents	Poor multi-step planning, dynamic web challenges	GPT-4: 14.41%, Human: 78.24%	Single-Agent System
AssistantBench [10]	Email drafting, customer data retrieval	LLMs	Hallucination, poor web navigation	<26% (complex web tasks)	Single-Agent System
Thakkar & Yadav [14]	Personalized recommendations, multimodal inputs	LLaMA-70B, Gemini-1.5-pro	Broad agent roles, lacks granular task decomposition	Recall@K: 1.0, ROUGE-1: 0.5042	Multi-Agent System

Table 1 shows that most prior approaches rely on single-agent systems, achieving low accuracies (3%–55%) due to limitations in contextual understanding, multi-step task execution, and web navigation [3], [8], [9], [10]. For example, CRMArena [3] reports success rates below 55% for tasks like Knowledge Question Answering (KQA), Named Entity Disambiguation (NED), and Policy Violation Identification (PVI), attributed to difficulties in handling nuanced contexts and integrating with CRM interfaces. WorkArena [8] and WebArena [9] similarly highlight poor performance of open-source models in enterprise and web-based tasks, limited by weak reasoning and planning capabilities. AssistantBench [10] notes issues like hallucination, with complex tasks achieving less than 26% accuracy. In contrast, Thakkar and Yadav [14] use a multi-agent system for personalized recommendations, achieving high recall but facing challenges with granular task decomposition, which limits effectiveness in complex CRM workflows [14].

These findings highlight a performance gap in open-source LLMs compared to proprietary models, which offer higher accuracy but are cost-prohibitive for startups and small-to-medium enterprises (SMEs) [3]. Multi-agent frameworks, leveraging architectures like LangGraph, show promise by enabling task specialization and collaborative problem-solving [7]. This literature review establishes a foundation for advancing open-source LLMs to achieve cost-effective, reliable e-commerce CRM automation.

Chapter 3

Methodology

3.1 Introduction

This chapter outlines the methodology employed in the research to design, implement, and evaluate a multi-agent system (MAS) integrating Large Language Models (LLMs) for simulated Customer Relationship Management (CRM) tasks within the CRMArena sandbox environment. The methodology encompasses data collection, data analysis, design, implementation, and evaluation phases. Each phase is described in detail, ensuring a structured and rigorous approach to achieving the research objectives. Figure 1 illustrates the workflow of the methodology.



Figure 1: Methodology Overview Diagram

3.2 Data Collection

The data collection phase uses the CRMArena sandbox, a publicly available dataset developed by Salesforce AI Research to simulate real-world Customer Relationship Management (CRM) environments [3]. Figure 2, the CRMArena logo, represents the benchmark's identity as a standardized platform for testing LLM-based CRM automation, guiding the selection of tasks and data for this research. This section describes the CRMArena sandbox characteristics, task design, selection of specific tasks and objects, access protocols, validation processes, and ethical considerations.



Figure 2: CRMArena Logo

3.2.1 Data Description

The CRMArena sandbox [3], a publicly available dataset, serves as the primary data source, replicating enterprise CRM systems with synthetic, realistic data. The dataset models a mid-sized fictional shoe company and includes:

- **Objects:** 16 Salesforce objects, such as Contact, Account, Case, Order, Product2, and Knowledge__kav, capturing e-commerce interactions.
- **Records:** Thousands of entries (e.g., 2,069 orders, 977 cases, 501 products) spanning a 4-year period, ensuring task complexity. Table 2 refers the number of entries per object.

Table 2: The number of entries per object.

Object	Number of Entries
User	253
Account	201
Contact	208
Case	977
Knowledge__kav	44
ProductCategory	12
Product2	501
ProductCategoryProduct	762
Pricebook2	46
PricebookEntry	22000
Order	2069
OrderItem	7100
EmailMessage	3241
LiveChatTranscript	774
Issue__c	10
CaseHistory__c	2983

- **Fields:** Each object contains relevant fields (e.g., Order: OrderID, Date, Amount; Case: CaseID, Status; Product: ProductID, Name, Price), supporting tasks like Named Entity Disambiguation (NED), Policy Violation Identification (PVI), and Knowledge-based

Question Answering (KQA). The detailed CRMArena schema, including all objects and fields, is provided in Appendix A.

- **Dependencies:** Objects exhibit high interconnectivity (e.g., Case linked to Account and Contact), as detailed in the schema diagram provided in Appendix B, Figure 13: CRMArena Sandbox Schema Diagram.

3.2.2 Data Collection Method

This subsection describes the techniques used to collect data from the CRMArena sandbox [3] for evaluating the multi-agent system (MAS) in simulated Customer Relationship Management (CRM) tasks. The following methods ensured efficient access to the dataset:

- **Access:** The CRMArena dataset was downloaded from its public repository (<https://github.com/SalesforceAIResearch/CRMArena>) [3] using provided authentication credentials (username and password).
- **Integration:** The simple-salesforce Python library was used to establish a client connection to the CRMArena dataset, enabling retrieval of data for CRM tasks. Figure 3 illustrates the implementation of the Salesforce client connection.

```
from simple_salesforce import Salesforce

sf_client = Salesforce(
    username='kh.huang+00dws000004urq4@salesforce.com',
    password='crmarenatest',
    security_token='ugvBSBv0ArI7dayfqUY0wMGu',
)
```

Figure 3: Salesforce client connection code

3.2.3 Data Validation

To ensure the reliability and suitability of the CRMArena dataset, the following validation processes were applied, prioritizing privacy concerns and data quality:

1. **Privacy and Synthetic Data:** The CRMArena dataset addresses privacy concerns by using synthetic data instead of real enterprise data, avoiding risks associated with personally identifiable information (PII) [3]. A scalable data generation pipeline, as described in the CRMArena benchmark, produces diverse and realistic CRM data across various domains, ensuring the dataset mirrors real-world e-commerce scenarios without compromising privacy.

2. **Integrity Checks:** Python-based scripts verified referential integrity (e.g., valid foreign keys between Account and Order) and confirmed no missing or malformed entries, ensuring data consistency.
3. **Query Testing:** Executed 50 sample queries per task to confirm data availability for CRM tasks, such as product identification, case-policy matching, and article retrieval.

These validation steps confirmed the dataset’s quality and appropriateness for evaluating the MAS in the CRMArena environment.

3.3 Data Analysis

This section details the preparation of the CRMArena dataset [3] for evaluating the multi-agent system (MAS) in simulated Customer Relationship Management (CRM) tasks, specifically Named Entity Disambiguation (NED), Policy Violation Identification (PVI), and Knowledge-based Question Answering (KQA). The CRMArena sandbox provides a well-defined dataset with structured objects, records, fields, and dependencies, which were processed to ensure compatibility with the MAS. Salesforce Workbench (<https://workbench.developerforce.com/query.php>) was utilized to support query testing and data extraction, enhancing the preparation process for efficient task execution in an e-commerce CRM environment.

3.3.1 Data Preparation

The CRMArena dataset’s structured design, as detailed in the schema provided in Appendix A, facilitated systematic preparation for the MAS. The preparation process involved the following steps, tailored to the requirements of NED, PVI, and KQA tasks:

3.3.1.1 Query Selection

- A total of 390 natural language queries (130 per task) were selected from the CRMArena dataset to test the MAS, reflecting real-world e-commerce scenarios for customer-facing CRM tasks.
- **NED Queries:** Focused on resolving ambiguous product names, e.g., “Can you display the women’s trail jacket I purchased a fortnight ago?” These queries required identifying specific products from customer purchase histories.

- **PVI Queries:** Aimed at detecting policy breaches, e.g., “Did the agent breach the policy in this situation?” These tested the system’s ability to compare interaction details against policy rules.
- **KQA Queries:** Targeted answering customer questions using knowledge articles, e.g., “What technology features do Shoes & Clothing’s golf shoes offer?” These assessed the retrieval of relevant information.
- Each task included 39 non-answerable queries (30% of the total), such as those referencing non-existent products, to simulate realistic CRM challenges. Salesforce Workbench was used to manually test sample queries, ensuring their alignment with the dataset’s structure.

3.3.1.2 Object and Record Mapping

Relevant CRM Arena objects and records were mapped to each task, leveraging the dataset’s structured objects and fields, as defined in the schema provided Appendix A. The primary objects and fields were:

- **NED:**
 - **Objects:** Order (ID, AccountId, EffectiveDate,...), OrderItem (ID, OrderId, Product2Id,...), Product2(ID, Name, Description,...), Account (ID, FirstName, LastName...) Contact (ID, AccountID, Email,...).
 - **Purpose:** Linked customer queries to purchase history and product details. For example, for the query “women’s trail jacket,” the system retrieved Order records by ContactID to identify the ProductID.
 - **Records:** Utilized 2,069 Order records, 501 Product2 records, and 208 Contact records to ensure sufficient data volume.
- **PVI:**
 - **Objects:** Case (ID, Description, IssueId_c, Subject,...), Knowledge__kav (ID, FAQ_Answer_c, Summary, Title,...)
 - **Purpose:** Compared case details with policy rules to detect violations. For instance, Case records were matched with Knowledge__kav articles to check compliance.
 - **Records:** Used 977 Case records, 44 Knowledge__kav articles, and 253 User records for policy analysis.

- **KQA:**
 - **Objects:** Knowledge__kav (ID, FAQ_Answer_c, Summary, Title, UrlName)
 - **Purpose:** Retrieved articles to answer queries about product features, ensuring accurate responses.
 - **Records:** Relied on 44 Knowledge__kav articles and 501 Product2 records for comprehensive coverage.

Salesforce Workbench facilitated the exploration of object relationships and record counts, verifying the availability of relevant data for each task.

3.3.1.3 Dependency Handling

For the NED task, the CRMArena dataset’s interconnectivity, with an average of 1.31 dependencies per object (illustrated in Appendix B, Figure 13: CRMArena Sandbox Schema Diagram), was leveraged to enable accurate data retrieval. For example, to resolve queries like “women’s trail jacket purchased a fortnight ago,” the MAS joined Contact (AccountId) and Order (AccountId) using SOQL queries, as detailed in the Design section. Example queries are shown in figures 4 and 5

The query in the figure 4 retrieves the AccountId from a Contact record.

```
SELECT AccountId FROM Contact WHERE Id = '{contact_id}'
```

Figure 4: Retrieve AccountId from Contact

- **Purpose:** This query retrieves the AccountId associated with a given Contact record, identified by a unique contact_id.
- **Dependency Handling:** The Contact object has a foreign key relationship with the Account object via AccountId. This query resolves the dependency by fetching the AccountId, which serves as a link to other related objects (e.g., Order).

The query in the figure 5 retrieves Product2 information from OrderItem records associated with the AccountId obtained from the first query

```
SELECT Product2Id, Product2.Name
FROM OrderItem
WHERE Order.AccountId = '{account_id}'
AND Order.EffectiveDate >= {effective_date}
```

Figure 5: Retrieve ProductID and Name from OrderItem

- Purpose: This query retrieves the Product2Id and Product2.Name from the OrderItem object, filtered by the AccountId (obtained from the first query) and a time-based condition (EffectiveDate).
- Dependency Handling: This query handles two dependencies:
 - Order to Account: The Order object is linked to the Account object via Order.AccountId. The query uses the account_id from the first query to filter OrderItem records associated with the correct account.
 - OrderItem to Product2: The OrderItem object references the Product2 object via Product2Id, allowing the query to retrieve product details (e.g., Product2.Name for “women’s trail jacket”).
 - Time Dependency: The EffectiveDate >= {effective_date} clause ensures the query retrieves orders from a specific timeframe (e.g., “a fortnight ago”), resolving temporal dependencies in the query.

Salesforce Workbench was used to test these dependency-driven queries, confirming the integrity of relationships (e.g., valid foreign keys) before processing by the MAS. Dependency handling was not required for PVI and KQA tasks, as their data retrieval relied on direct access to Case and Knowledge__kav.

3.4 Design

This section presents the design of a modular, intelligent system to automate complex Customer Relationship Management (CRM) tasks—Named Entity Disambiguation (NED), Policy Violation Identification (PVI), and Knowledge-based Question Answering (KQA)—within the CRMArena sandbox [3]. The system leverages a multi-agent architecture orchestrated by LangGraph to ensure flexibility, scalability, and task-specific processing.

3.4.1 Objective of the Design

The design aims to:

- Implement a Task Classifier to route queries to the appropriate workflow automatically.
- Develop task-specific agent workflows for NED, PVI, and KQA to handle complex, multi-step CRM queries.

- Optimize prompt engineering and agent collaboration to improved accuracy and efficiency.

3.4.2 System Architecture

The system employs a multi-agent graph-based architecture orchestrated by LangGraph, ensuring modularity and scalability. Key components include:

- **Task Classifier Agent:** Identifies the task type (NED, PVI, KQA) based on query structure and keywords, routing the query to the corresponding task agent using a prompt-based LLM classifier.
- **Task Agents:** Specialized agents for NED, PVI, and KQA, each with sub-agents handling individual steps of the task.
- **Query Tools:** Utilize Salesforce Object Query Language (SOQL) for structured data retrieval and Salesforce Object Search Language (SOSL) for text-based searches.
- **State Management:** Each agent maintains its own WorkflowState to track query data, intermediate results, and errors specific to its workflow, ensuring isolated and task-specific state handling.

3.4.2.1 Rationale for Design Choices

The architecture leverages a multi-agent approach and LangGraph for the following reasons:

- **Multi-Agent Approach:** By assigning distinct agents to NED, PVI, and KQA, the system modularizes task-specific logic, improving reusability and debugging compared to a monolithic system. This approach supports CRMarena’s complex, multi-step query requirements [3].
- **LangGraph:** Chosen for its low-level flexibility, LangGraph enables explicit control over state transitions, conditional routing, and retry logic. Unlike other frameworks (e.g., CrewAI, AutoGen), it provides transparent debugging, separate state handling for each agent, and graph-based workflows, making it ideal for orchestrating multi-step CRM tasks.

3.4.3 Agents and Sub-Agents

3.4.3.1 Task Classifier Agent

Figure 6 illustrates the process of classifying user queries into NED, PVI, or KQA tasks based on structure and keywords.

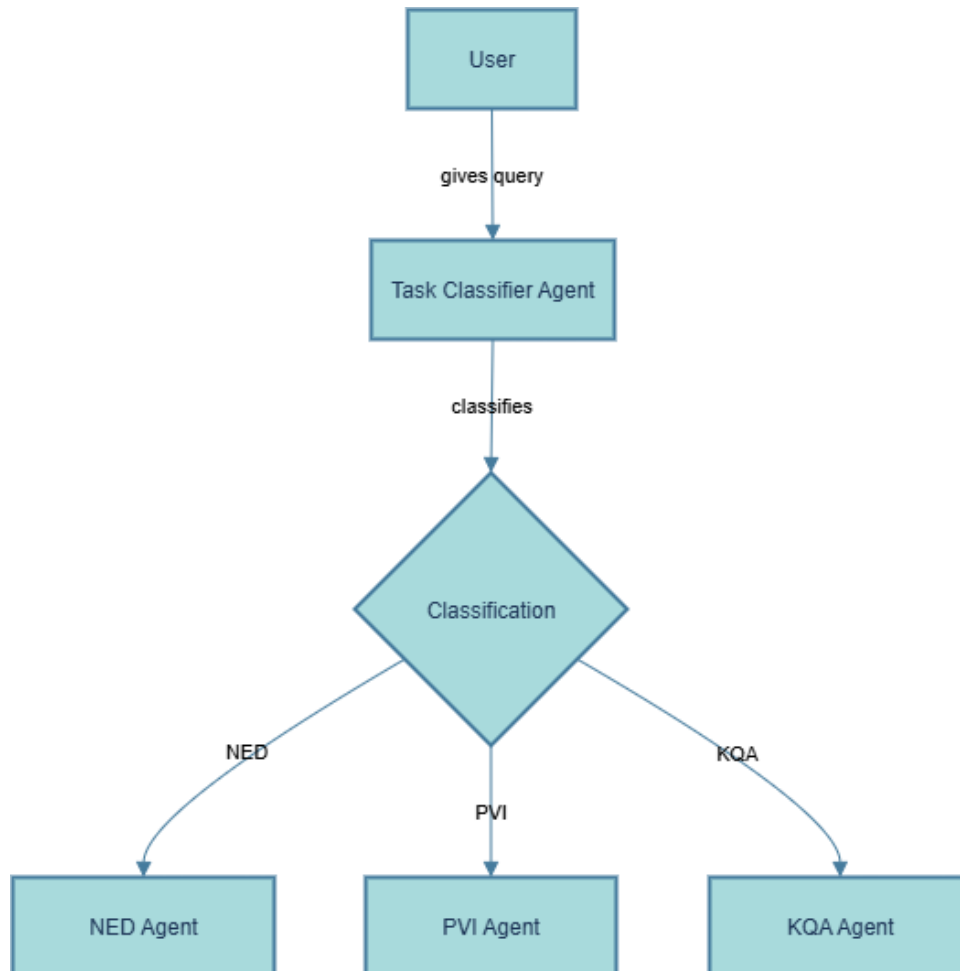


Figure 6: Task Classifier Agent Workflow Diagram

- **Responsibility:** Analyzes query structure and keywords to classify it as NED, PVI, or KQA, routing it to the appropriate Task Agent, as shown in Figure 3.
- **Workflow:** Uses a prompt-based LLM classifier to identify task type based on intent (e.g., product purchases for NED, policy questions for PVI, general queries for KQA).

3.4.3.2 NED Agent

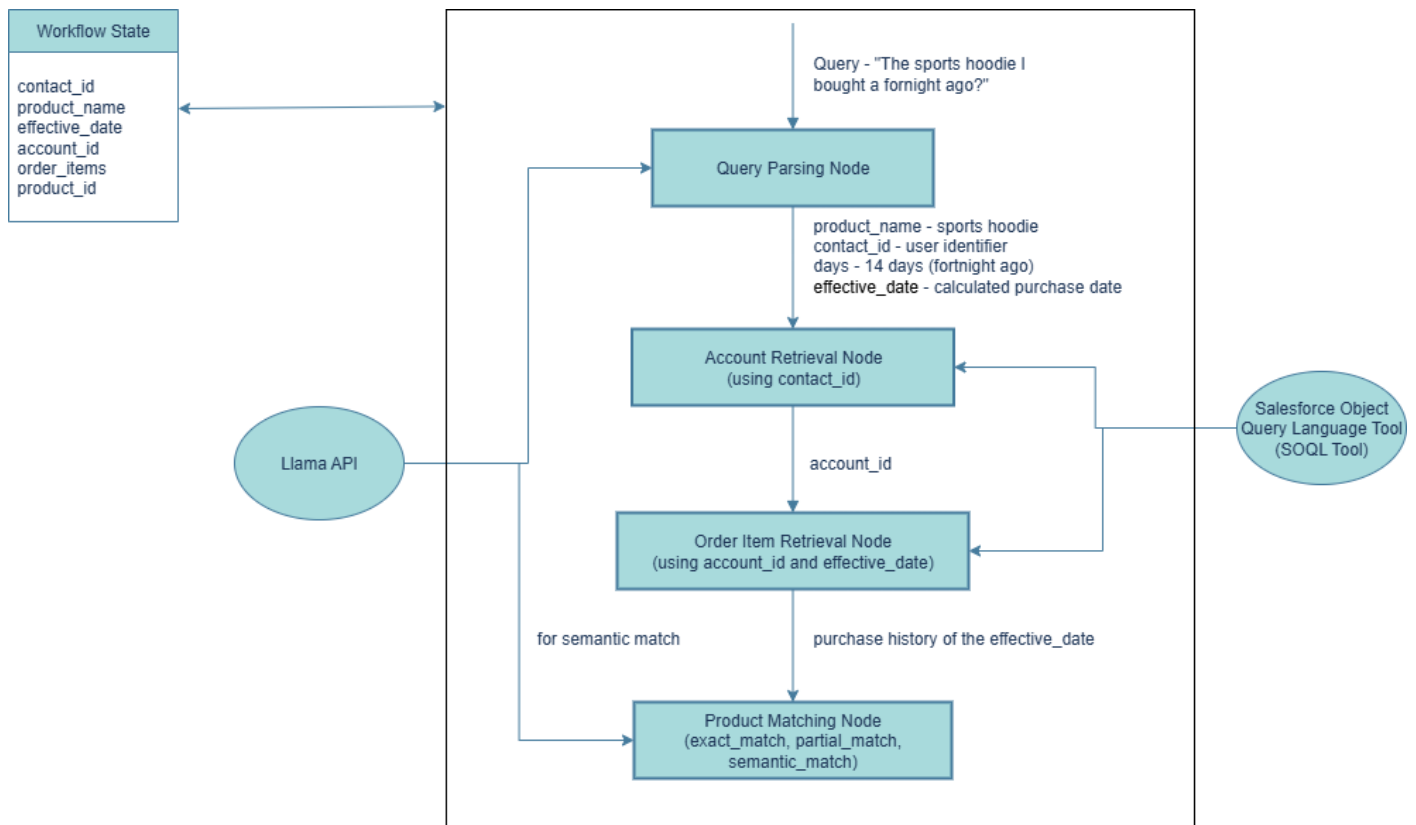


Figure 7: NED Agent Workflow Diagram

Figure 7 details the steps for resolving ambiguous product names in NED queries.

- **Responsibility:** Resolves ambiguous product names in queries (e.g., “women’s trail jacket I purchased a fortnight ago”) to a unique ProductID, as depicted in Figure 4.
- **Sub-Agents:**
 - **Query Parsing:** Extracts product name and time window (e.g., “fortnight ago” as 14 days).
 - **Account Retrieval:** Retrieves AccountId from Contact using SOQL.
 - **OrderItem Retrieval:** Fetches OrderItem records within the time range.
 - **Product Matching:** Matches product names using exact, partial, or semantic matching.

3.4.3.3 PVI Agent

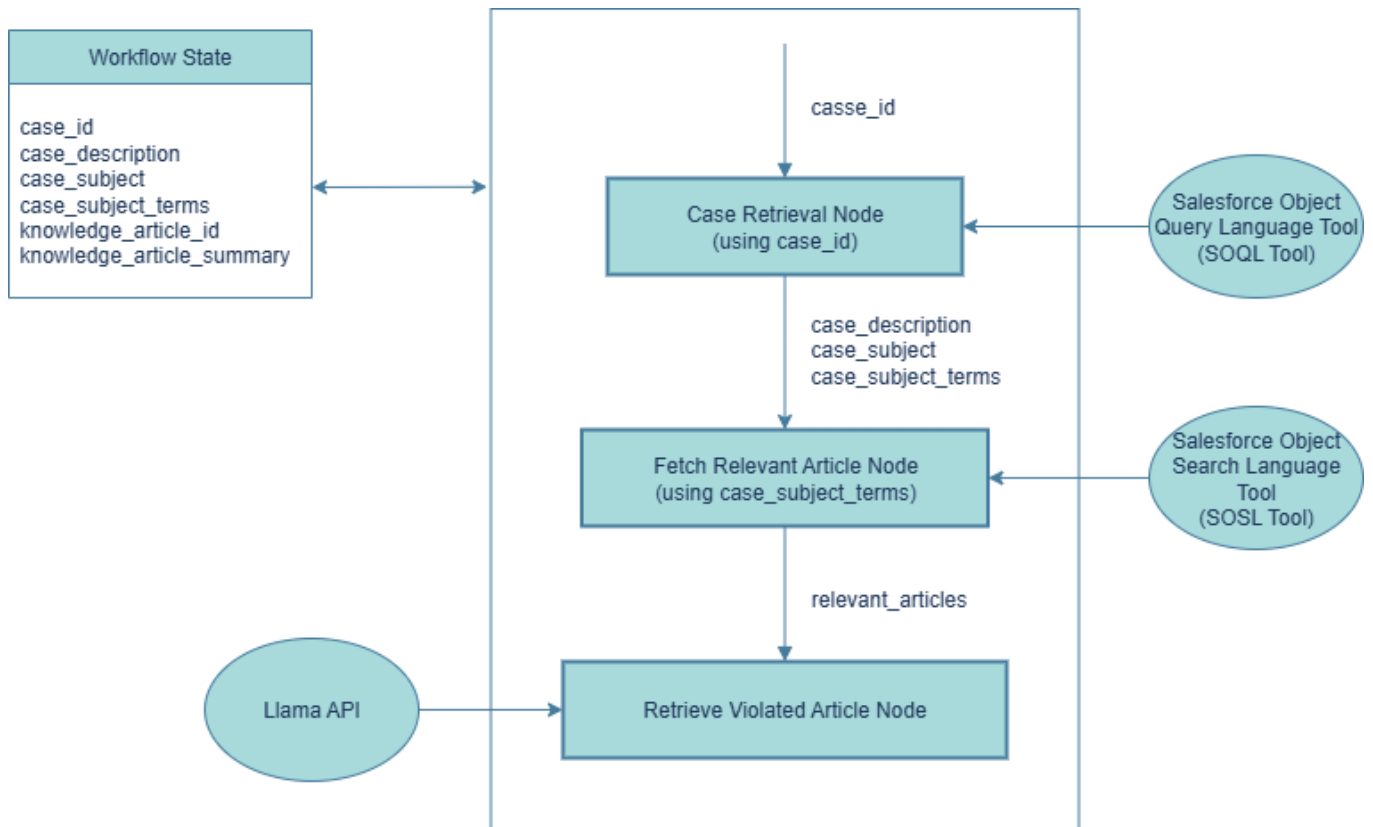


Figure 8: PVI Agent Workflow Diagram

Figure 8 outlines the process for identifying policy violations in customer service cases.

- **Responsibility:** Identifies policy violations in customer service cases by comparing case details with knowledge articles, as shown in Figure 5.
- **Sub-Agents:**
 - **Case Retrieval:** Retrieves case details (e.g., Description) using SOQL.
 - **Article Retrieval:** Searches Knowledge__kav for relevant articles using SOSL.
 - **Violation Check:** Uses an LLM to match case details with articles, returning “None” if no violation is found.

3.4.3.4 KQA Agent

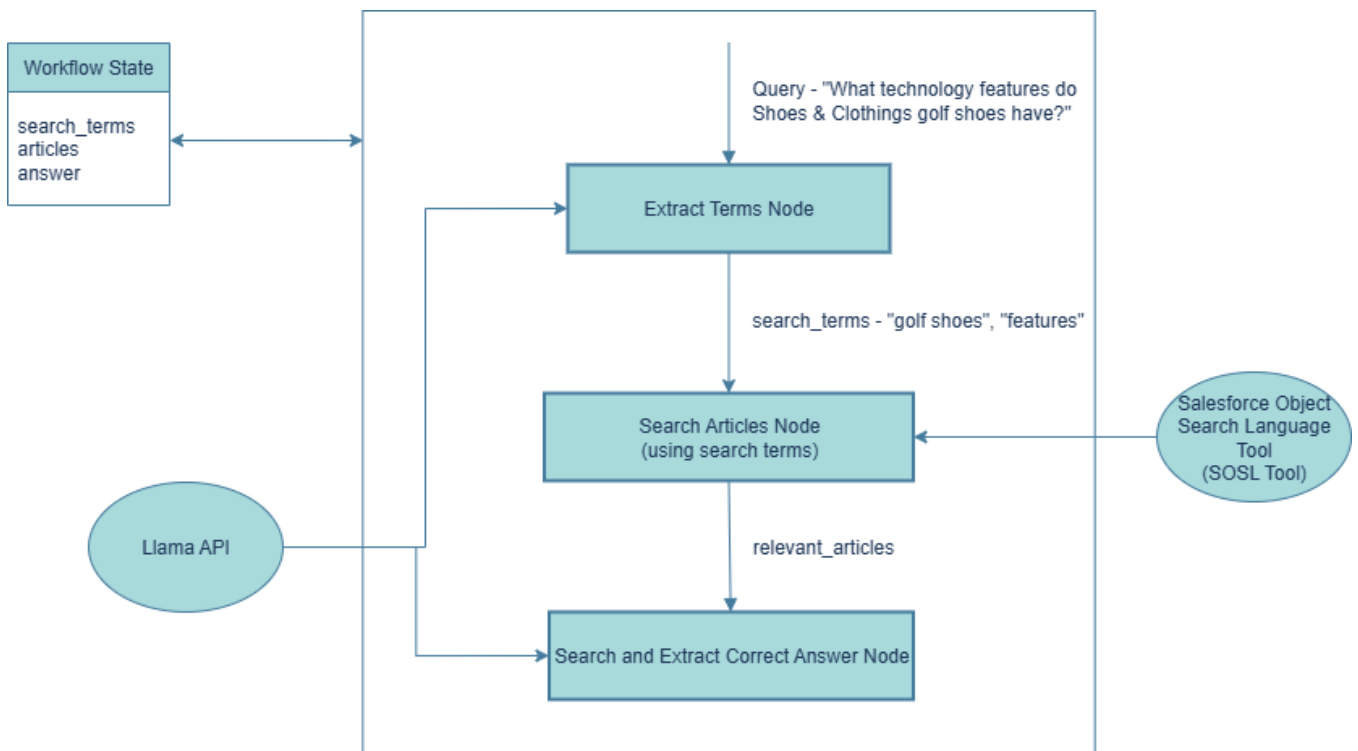


Figure 9: KQA Agent Workflow Diagram

Figure 9 illustrates the steps for answering general customer queries using knowledge articles.

- **Responsibility:** Answers general customer queries (e.g., “What technology features do Shoes & Clothing’s golf shoes offer?”) using knowledge articles, as depicted in Figure 6.
- **Sub-Agents:**
 - **Search Term Extraction:** Extracts key terms from queries for SOSL searches.
 - **Article Retrieval:** Searches Knowledge__kav for relevant articles.
 - **Answer Extraction:** Generates concise answers from article content.

3.5 Implementation

This section describes the practical implementation of the multi-agent system for automating Customer Relationship Management (CRM) tasks—Named Entity Disambiguation (NED), Policy Violation Identification (PVI), and Knowledge-based Question Answering (KQA)—within the CRMArena sandbox [3]. The implementation served two purposes: evaluation of the system using Google Colab to test 390 queries (130 per task) and development of an interactive prototype using Django and React. It details the tools, libraries, code structure, LLaMA-3.1 integration, agent connections via LangGraph, and challenges faced during implementation.

3.5.1 Environment Setup

A robust environment was established to support development, testing, and demonstration of the multi-agent system:

3.5.1.1 Tools and Libraries

- **Python 3.12:** Core language for backend logic and query processing.
- **LangGraph:** Framework for orchestrating multi-agent workflows.
- **simple-salesforce:** Python library for executing Salesforce Object Query Language (SOQL) and Salesforce Object Search Language (SOSL) queries to access CRMArena data.
- **SOQL (Salesforce Object Query Language):** Used for structured data retrieval from specific Salesforce objects (e.g., Contact, OrderItem).
- **SOSL (Salesforce Object Search Language):** Enabled full-text searches across multiple objects (e.g., Knowledge__kav).
- **Pydantic:** Used for defining WorkflowState models to manage agent states.
- **Django 5.2.1 and django-rest-framework:** Backend framework for API endpoints in the prototype.
- **Node.js, Vite, React, Tailwind CSS:** Frontend tools for building the interactive prototype (provided in Appendix C.3 Prototype UI screenshots).
- **DeepInfra:** Cloud platform for accessing LLaMA-3.1 models (70B for development, 405B for final testing).
- **Google Colab:** Platform for batch testing 390 queries (130 per task: NED, PVI, KQA).

3.5.1.2 Setup Steps

1. Installed Python libraries (langgraph, simple-salesforce) in a virtual environment for both Colab and prototype development.
2. Configured Node.js, Vite, and React with Tailwind CSS for the prototype's frontend.
3. Set up DeepInfra API keys to access LLaMA-3.1 models in Colab and Django.
4. Initialized a simulated Salesforce environment using the CRMarena dataset, accessing objects like Contact, Case, and Knowledge__kav.
5. Configured LangGraph to define agent nodes and connections for both evaluation and prototype workflows.
6. Set up Django REST API endpoints to connect agents to the React frontend for the prototype.
7. Configured Google Colab with the same Python libraries for batch testing.

3.5.2 Code Structure

The implementation involved distinct code structures for evaluation in Google Colab and prototype development using Django and React, tailored to the CRMarena tasks:

3.5.2.1 Evaluation in Google Colab

- **Purpose:** Designed to test 390 queries (130 per task: NED, PVI, KQA) to validate the system's functionality in a controlled environment.
- **Code Organization:**
 - **NED Workflow:** Implemented as a sequence of functions for sub-agents (Query Parsing, Account Retrieval, OrderItem Retrieval, Product Matching). SOQL queries (provided in Appendix C.2.1 NED SOQL Query for Account Retrieval, C.2.2 NED SOQL Query for OrderItem Retrieval) retrieved data from Contact and OrderItem objects, with semantic matching handled by LLaMA-3.1 via DeepInfra.
 - **PVI Workflow:** Coded to retrieve case details and match them with Knowledge__kav articles using SOSL queries (provided in Appendix C.2.3 PVI SOQL Query for Case Retrieval, C.2.4 PVI SOSL Query for Filter relevant article) and LLaMA-3.1 for violation detection.
 - **KQA Workflow:** Structured to extract search terms, perform SOSL searches on Knowledge__kav (provided in Appendix C.2.5 KQA SOSL Query), and generate answers using LLaMA-3.1.

- **Batch Processing:** A main script iterated over 390 queries, calling task-specific functions and storing intermediate results in WorkflowState models for debugging.
- **Implementation Details:** Used simple-salesforce for SOQL/SOSL queries, validated via Salesforce Workbench. Queries were processed sequentially to ensure accurate state management, with LLaMA-3.1 (70B) used for development and 405B for final tests.

3.5.2.2 Prototype Development

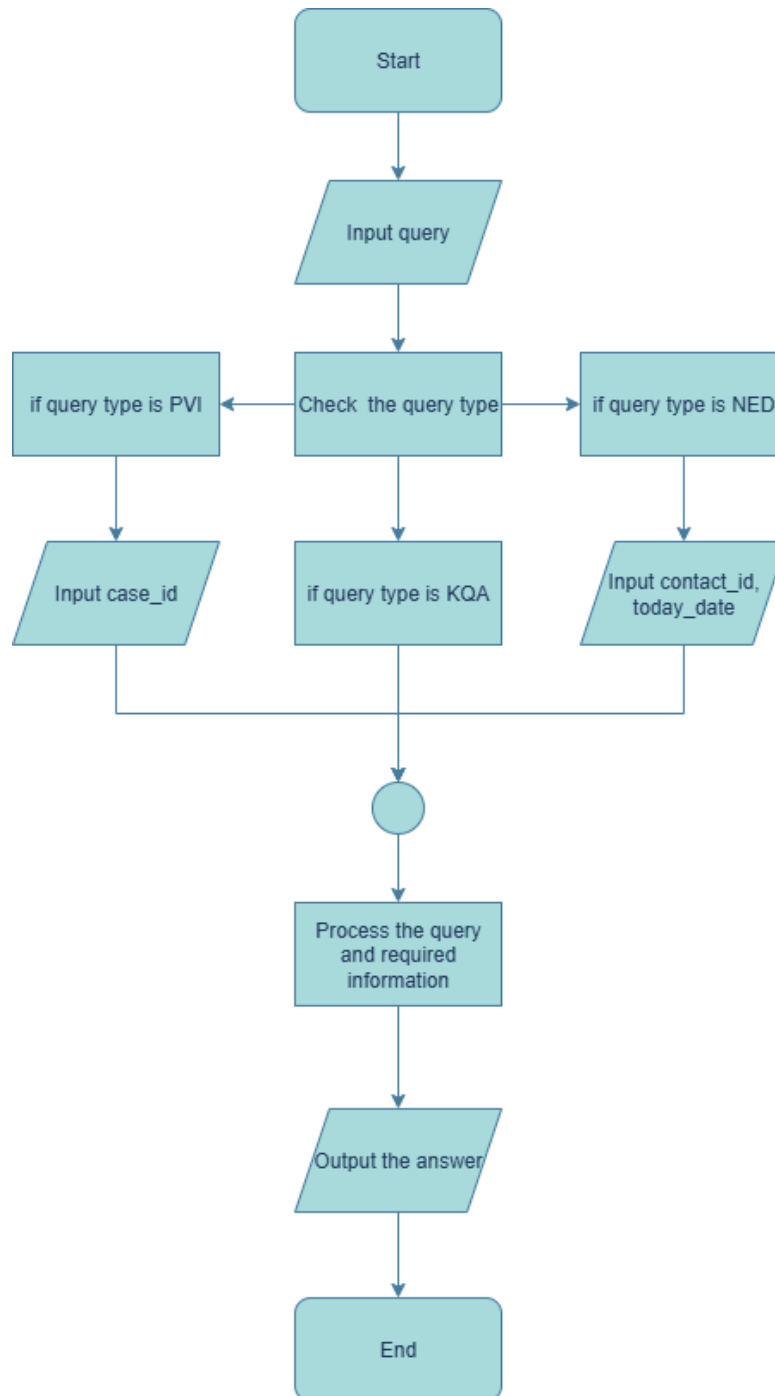


Figure 10: Prototype Flowchart Diagram

The Prototype Flowchart Diagram, Figure 10 illustrates the workflow of the interactive interface designed to demonstrate agent workflows. The purpose of the prototype is to allow users to submit queries and visualize the processing steps, with the flowchart likely depicting the sequence and interaction of components like the Task Classifier, NED, PVI, and KQA workflows. The diagram, as referenced, would show how queries are routed and processed through these agents, with transitions managed by LangGraph nodes and edges.

Key Components of the Prototype

- **Task Classifier Code:** A Python script implemented the Task Classifier Agent using a prompt-based LLaMA-3.1 classifier (provided in Appendix C.1.1 Task Classifier Prompt) to route queries to NED, PVI, or KQA workflows. The script processed queries in batches, storing task types in a Pydantic WorkflowState model.
- **Django Backend:** Implemented REST API endpoints to handle query processing. Each endpoint called LangGraph nodes for Task Classifier and Task Agents, using simple-salesforce for data access and DeepInfra for LLaMA-3.1 inference.
- **React Frontend:** Developed components (e.g., TaskClassifier.jsx, NedTask.jsx) using Vite and Tailwind CSS to display query inputs and agent outputs (Appendix C.3 Prototype UI screenshots). API calls to Django endpoints retrieved results for display.
- **LangGraph Integration:** Defined nodes for each agent and sub-agent, with edges controlling workflow transitions (e.g., from classification to task-specific processing).
- **State Management:** Used Pydantic WorkflowState models to track query data and errors, serialized for API responses.

3.5.3 LLaMA-3.1 Integration

LLaMA-3.1 models (70B and 405B) were integrated via DeepInfra's API for efficient inference:

- **Loading Models:** Configured DeepInfra API keys in Google Colab for evaluation and Django for the prototype. The 70B model was used during development for faster iteration, while the 405B model was employed for testing the accuracy.
- **Usage:** LLaMA-3.1 powered the Task Classifier (query classification), NED Agent (semantic matching), PVI Agent (violation detection), and KQA Agent (answer generation). Prompts were optimized for structured outputs (Appendix C.1 Prompts).

- **Code Implementation:** In Colab, Python scripts called DeepInfra APIs for batch query processing. In the prototype, Django views invoked LLaMA-3.1 via DeepInfra for real-time responses, with results cached to reduce latency.

3.5.4 Agent Connection via LangGraph

Agents were connected using LangGraph to form a graph-based workflow for both evaluation and prototype:

- **Task Classifier Node:** Implemented as a Python function in Colab and a Django endpoint (`classify_task`) in the prototype, using the prompt in Appendix C.1.1 Task Classifier Prompt to route queries to NED, PVI, or KQA nodes.
- **Task Agent Nodes:** Each task (NED, PVI, KQA) had dedicated LangGraph nodes with sub-nodes for sub-agents (e.g., Query Parsing, Account Retrieval for NED). Edges defined transitions (e.g., from parsing to retrieval), implemented in Python scripts for Colab and Django views for the prototype.

Query Tools: Integrated simple-salesforce for SOQL/SOSL queries (provided in Appendix

- C.2 SOQL / SOSL Queries), with results stored in WorkflowState models. Salesforce Workbench validated queries during development.
- **Frontend Integration:** In the prototype, Django API endpoints sent agent outputs to React components, displaying results in a user-friendly interface (provided in Appendix C.3 Prototype UI screenshots).

3.5.5 Integration Challenges

Several challenges arose during implementation:

- **Query Routing Accuracy:** The Task Classifier misclassified vague queries, requiring prompt refinements (as shown in Appendix C.1.1 Task Classifier Prompt) to improve keyword detection.

SOQL/SOSL Complexity: Complex object dependencies (e.g., Contact-to-Order joins) caused query errors, addressed by iterative testing in Salesforce Workbench (provided in Appendix

- C.2 SOQL / SOSL Queries).

- **LLM Variability:** LLaMA-3.1 outputs varied for semantic matching in NED and answer generation in KQA, mitigated by structured prompts and validation logic.
- **Frontend-Backend Sync:** Synchronizing Django API responses with React updates was challenging, resolved by optimizing endpoints and caching results.
- **Scalability in Colab:** Processing 390 queries in Google Colab was resource-intensive, addressed by batching queries and using the 405B model for final tests.

These implementation steps enabled a functional evaluation environment in Google Colab and an interactive prototype, addressing CRMArena’s challenges for reliable CRM task automation.

3.6 Evaluation

This section details the methodology for evaluating the performance of the multi-agent system designed for automating Customer Relationship Management (CRM) tasks—Named Entity Disambiguation (NED), Policy Violation Identification (PVI), and Knowledge-based Question Answering (KQA)—within the CRMArena sandbox [3]. The evaluation was conducted exclusively in Google Colab, with each task evaluated separately using 390 queries (130 per task) from the CRMArena dataset. Outputs were tested against ground truth, with accuracy calculated via exact match for NED and PVI, and fuzzy match using BERTScore for KQA. Performance of LLaMA-3.1 models (70B and 405B) was compared for accuracy and inference cost via DeepInfra.

3.6.1 Purpose of Evaluation

The evaluation aimed to measure the system’s accuracy and inference cost for NED, PVI, and KQA tasks, comparing performance against the CRMArena benchmark accuracy of 55% [3]. It validated the system’s ability to process diverse CRM queries independently for each task and assessed the trade-offs between the LLaMA-3.1 70B and 405B models in terms of accuracy and computational cost.

3.6.2 Test Dataset and Setup

- **Test Dataset:** A total of 390 queries from the CRMArena dataset were used, with 130 queries per task, evaluated separately:
 - **NED:** Queries identifying purchased products (e.g., “Who bought Men’s Trail Jacket?” or “Display the waterproof hiking boots I purchased three days back”).

- **PVI:** Queries checking policy violations (e.g., “Check case for policy violation” or “Did the agent breach the policy in this instance?”).
- **KQA:** General knowledge questions (e.g., “What is the return policy?” or “What technology features do Shoes & Clothings golf shoes have?”).
- **Setup:**
 1. Configured Google Colab with Python libraries (langgraph, simple-salesforce, pydantic) and DeepInfra API access for LLaMA-3.1 models (70B for development, 405B for final testing).
 2. Initialized a simulated Salesforce environment using the CRMarena dataset, accessing objects like Contact, Case, OrderItem, and Knowledge__kav.
 3. Implemented separate LangGraph workflows for each task (NED, PVI, KQA), using task-specific prompts and SOQL/SOSL queries (Appendix C.1 Prompts, C.2 SOQL / SOSL Queries).
 5. Processed queries in batches for each task, ensuring independent evaluation to avoid cross-task interference.

3.6.3 Evaluation Metrics

The following metrics were measured for each task independently

- **Accuracy:** Percentage of correct outputs compared against ground truth:
 - **NED:** Exact match between the predicted ProductID and the ground truth ProductID in the CRMarena dataset (e.g., matching “003Ws000004Fo3qIAC” for “women's trail jacket”).
 - **PVI:** Exact match between the predicted violation status (article ID like “ka0Ws000000QriDIAS” or “None”) and the ground truth status.
 - **KQA:** Fuzzy match using BERTScore to assess semantic similarity between the predicted answer and the ground truth answer from Knowledge__kav articles. An ($F1 \geq 0.85$) was used as the threshold for correctness.
 - **Formula for Accuracy**
For NED and PVI, a correct output requires an exact match with the ground truth. For KQA, a correct output requires a BERTScore ($F1 \geq 0.85$). The formulas used are shown in figures 11 and 12.

$$\text{Accuracy}(\%) = \left(\frac{\text{Number of Correct Outputs}}{\text{Total Queries}} \right) \times 100$$

Figure 11: Accuracy formula for NED and PVI

An output is considered correct only if it exactly matches the ground truth.

- **BERTScore for KQA:** BERTScore evaluates semantic similarity using contextual embeddings from a BERT model, computing precision (P), recall (R), and F1-score (F1)

$$P = \frac{1}{|A|} \sum_{a_i \in A} \max_{g_j \in G} \cos(e_{a_i}, e_{g_j}), \quad R = \frac{1}{|G|} \sum_{g_j \in G} \max_{a_i \in A} \cos(e_{a_i}, e_{g_j}), \quad F1 = 2 \cdot \frac{P \cdot R}{P + R}$$

Figure 12: BERTScore formula for KQA

Where:

- A = predicted answer
- G = ground truth answer
- e_{ai}, e_{gi} = BERT embeddings
- cos = cosine similarity

A pre-trained BERT model (e.g., BERT-base-uncased) was used to generate embeddings, with ($F1 \geq 0.85$) indicating a correct KQA output.

- **Inference Cost:** The inference cost was calculated per task by measuring the total cost of running 130 queries for each task. The cost was determined based on DeepInfra's pricing for the LLaMA-3.1 70B and 405B models.

Chapter 4

Results

4.1 Introduction

This section presents the raw performance results of the multi-agent system for NED, PVI, and KQA tasks, evaluated using 390 queries (130 per task) in Google Colab with LLaMA-3.1 models (70B and 405B). Results include accuracy, inference cost, error logs, and example predictions.

4.2 Accuracy and Inference Cost

Table 3: Accuracy and Inference Cost for NED, PVI, and KQA

Task	Model	Accuracy (%)	Testing Cost (\$)
NED	LLaMA-3.1-70B	92.31 (120/130)	0.04
	LLaMA-3.1-405B	86.92 (113/130)	0.16
PVI	LLaMA-3.1-70B	59.23 (77/130)	0.07
	LLaMA-3.1-405B	36.15 (47/130)	0.25
KQA	LLaMA-3.1-70B	85.38 (111/130)	0.16
	LLaMA-3.1-405B	84.62 (110/130)	0.85

Table 3 presents the accuracy and inference cost for the LLaMA-3.1 70B and 405B models across NED, PVI, and KQA tasks, based on 130 queries per task. Accuracy is shown as the percentage of correct outputs (exact match for NED and PVI, BERTScore ($F1 \geq 0.85$) for KQA), and cost is the total DeepInfra API cost (\$) for processing each task’s queries.

4.3 BERTScore Results for KQA

Table 4 presents BERTScore F1 results for selected KQA queries, measuring the semantic similarity between the MAS's generated responses and ground-truth answers. Scores range from 0 to 1, with higher values indicating better alignment.

Table 4: BERTScore F1 Results for KQA Queries

Query ID	Query	Predicted Output	Ground Truth	F1 Score
2	Can I get a full refund for damaged running shoes after 30 days?	No, the request must be submitted within 30 days of the purchase date.	No, the request must be submitted within 30 days of the purchase date.	1.0000
8	What is the typical time window for cancelling an order at Shoes & Clothings?	typically within the first 24 hours	Within the first 24 hours.	0.9556
26	What options are available if I receive an incorrect item from Shoes & Clothings?	If you have received an item different from what you ordered, you have two primary options to address the situation: a Full Refund or a Replacement.	Full Refund or Replacement	0.8587
17	What is offered to customers to make their next shopping experience with Shoes & Clothings more favorable if the desired sneaker is out of stock?	At Shoes & Clothings, we understand how disappointing it can be when you're unable to find the highly sought-after sneaker model you've been eyeing	Free Shipping on Next Order.	0.8172

4.4 Example Predictions

Table 5: Sample Outputs for NED, PVI, and KQA

Task	Input	Predicted Output	Ground Truth	Correct ?
NED	Query - Can you display the basic fitness tracker band I purchased two weeks ago?	Product ID - 01tWs000002wRTJIA2	Product ID - 01tWs000002wRTJIA2	√
PVI	Case Id D - 500Ws000007qnYaIAI Case Description - The size chart provided on the website does not accurately represent the actual product size. Case Subject - Size Mismatch	Knowledge article ID - ka0Ws000000QriDIAS	Knowledge article ID - ka0Ws000000QriDIAS	√
KQA	What technology features do Shoes & Clothings golf shoes have?	Shoes & Clothings golf shoes feature advanced sole technology and waterproof materials to keep your feet dry and comfortable throughout your game.	Advanced sole technology and waterproof materials	√ (f1 = 88.07 > 85 %)

Table 5 show that the MAS accurately processed queries across all three tasks. For NED, the system correctly identified the Product ID (01tWs000002wRTJIA2) for the query about a fitness tracker band, matching the ground truth. For PVI, the MAS accurately retrieved the correct Knowledge Article ID (ka0Ws000000QriDIAS) for a case involving a size mismatch, aligning with the ground truth. For KQA, the system generated a response detailing advanced sole technology and waterproof materials for golf shoes, achieving a BERTScore F1 of 88.07, which exceeds the threshold of 85% established in the evaluation, indicating high semantic similarity to the ground truth. These examples demonstrate the MAS’s capability to handle customer-facing CRM tasks effectively using the CRMArena dataset’s structured data.

Chapter 5

Discussion

5.1 Introduction

This section interprets the findings from the multi-agent system (MAS) evaluation on the CRMArena dataset [3] for Named Entity Disambiguation (NED), Policy Violation Identification (PVI), and Knowledge-based Question Answering (KQA) tasks, as presented in Chapter 4. It discusses the MAS's performance, inference cost implications, semantic accuracy of KQA responses, and a comparison with prior studies, while addressing limitations in the research methodology to provide a comprehensive evaluation of the project's outcomes for e-commerce Customer Relationship Management (CRM) automation.

5.2 Interpretation of Performance Across Tasks

The results in Chapter 4, Table 3: Accuracy and Inference Cost for NED, PVI, and KQA) demonstrate varied MAS performance across NED, PVI, and KQA tasks using LLaMA-3.1 models (70B and 405B). The 70B model outperformed the 405B model in NED, achieving 92.31% accuracy (120/130 queries) compared to 86.92% (113/130), likely due to NED's reliance on straightforward object mappings. This suggests that the 70B model efficiently processed the CRMArena dataset's structured data for product identification tasks.

In contrast, PVI exhibited lower accuracies of 59.23% (70B, 77/130) and 36.15% (405B, 47/130), indicating challenges in detecting policy violations. The complexity of matching Case records to Knowledge__kav articles may have hindered performance, requiring nuanced reasoning that both models struggled to achieve.

KQA showed robust performance, with accuracies of 85.38% (70B, 111/130) and 84.62% (405B, 110/130) based on a BERTScore F1 threshold of ≥ 0.85 , reflecting high semantic accuracy for open-ended queries.

5.3 Inference Cost Implications

In Chapter 4, Table 3: Accuracy and Inference Cost for NED, PVI, and KQA) reveals significant cost differences between the models, with the 70B model incurring lower inference costs (e.g., \$0.04 for NED, \$0.07 for PVI, \$0.16 for KQA) compared to the 405B model (e.g., \$0.16 for NED, \$0.25 for PVI, \$0.85 for KQA). The 405B model's higher computational requirements increased costs without consistently improving accuracy (e.g., lower performance in NED and PVI). For small-to-medium enterprises (SMEs) in e-commerce, the 70B model's cost-effectiveness, particularly for NED (\$0.04, 92.31%), supports its viability for resource-constrained environments. However, the high cost of KQA for the 405B model (\$0.85) suggests that optimizing computational efficiency is critical for scaling MAS deployment, especially for tasks involving extensive text generation. These cost-performance trade-offs inform practical decisions for CRM automation, favouring the 70B model for cost-sensitive applications.

5.4 BERTScore Findings for KQA Task

The BERTScore F1 results for KQA in Chapter 4, Table 4: BERTScore F1 Results for KQA Queries) provide insights into the MAS's semantic accuracy for open-ended queries, with implications for customer support applications.

1. Key Observations:

- **High-Scoring Answers ($F1 \geq 0.90$):** Responses with F1 scores ≥ 0.90 showed strong alignment with ground-truth answers. For example, Q2 ($F1 = 1.0$) accurately stated, "No, the request must be submitted within 30 days," and Q8 ($F1 \approx 0.956$) correctly identified the 24-hour order cancellation window, demonstrating precise information retrieval.
- **Lower-Scoring Answers ($F1 < 0.85$):** Responses like Q17 ($F1 \approx 0.817$) lacked specificity, offering generic answers instead of "Free Shipping on Next Order," indicating limitations in capturing fine-grained details.
- **Model Performance:** Both models performed similarly for high-scoring queries (e.g., Q2, Q8), suggesting that model size had minimal impact on KQA accuracy at higher F1 thresholds.

2. Threshold Recommendation:

- A threshold of $F1 \geq 0.85$ is recommended for KQA responses, ensuring semantic closeness to the ground truth while allowing minor phrasing differences. Queries above this threshold (e.g., Q2, Q8, Q26) were factually correct, while those below (e.g., Q17) often missed critical details, aligning with the evaluation criteria and the KQA example in Table 5 ($F1 = 88.07$).

3. Actionable Insights:

- For high-stakes CRM applications (e.g., customer support), a threshold of $F1 \geq 0.90$ ensures maximum accuracy, minimizing errors in critical interactions.
- For general use, $F1 \geq 0.85$ balances accuracy and coverage, with manual review recommended for scores between 0.80–0.85 (e.g., Q17) to address borderline cases.

5.5 Comparison with Prior Studies

The MAS’s overall accuracy of 78.97% (LLaMA-3.1-70B) across NED, PVI, and KQA tasks was compared to prior studies, as summarized in Table 6.

Table 6: Comparison of Proposed System with Prior Studies

Study	Main Tasks	Key features	Limitations	Accuracy	System type
CRMArena [3]	Automates CRM tasks using manually crafted function calling, real-world simulation	LLMs (LLaMA-3.1, GPT-4), ReAct, Function-Calling	Low accuracy, struggles with context and multi-step tasks	<55%	Single-Agent System
WorkArena [8]	Enterprise software tasks, mimics knowledge workers	LLMs, Web-based agents	Open-source models underperform, limited reasoning	GPT-4: 43%, LLaMA2-70B: 3%	Single-Agent System
WebArena [9]	Realistic web tasks, e-commerce support	LLMs, Web navigation agents	Poor multi-step planning, dynamic web challenges	GPT-4: 14.41%, Human: 78.24%	Single-Agent System

AssistantBench [10]	Email drafting, customer data retrieval	LLMs	Hallucination, poor web navigation	<26% (complex web tasks)	Single-Agent System
Thakkar & Yadav [14]	Personalized recommendations, multimodal inputs	LLaMA-70B, Gemini-1.5-pro	Broad agent roles, lacks granular task decomposition	Recall@K: 1.0, ROUGE-1: 0.5042	Multi-Agent System
Proposed System	Task-specific agents, workflow decomposition, enhanced contextual understanding	LLaMA 3.1- 70B, 405B, LangGraph	Limited model diversity, narrow task scope, synthetic dataset constraints	78.97 % (70B) 69.23 % (405B)	Multi-Agent System

The MAS’s 78.97% accuracy significantly outperforms single-agent systems, including CRMArena (<55%), WorkArena (43% GPT-4, 3% LLaMA2-70B), WebArena (14.41% GPT-4), and AssistantBench (<26%), due to its task-specific agents, workflow decomposition, and LangGraph framework, which effectively navigate the CRMArena dataset’s complex object dependencies (provided in Appendix B, Figure 13: CRMArena Sandbox Schema Diagram). Compared to Thakkar & Yadav’s multi-agent system [14], the MAS’s higher accuracy (78.97% vs. ROUGE-1: 0.5042) reflects improved contextual understanding and task decomposition for NED and KQA, though PVI’s lower performance suggests challenges in complex reasoning, similar to limitations in prior studies.

5.6 Research Limitations

The research methodology has several limitations that impact the findings and their generalizability, as evaluated below:

1. **Limited Model Diversity:** The study used only LLaMA-3.1 models (70B and 405B), excluding other advanced LLMs (e.g., GPT-4, used in CRMArena [3] and WorkArena [8]). This restricts comparisons with broader LLM capabilities, potentially underestimating the MAS’s performance with more advanced models.
2. **Narrow Task Scope:** The evaluation focused on three CRMArena tasks (NED, PVI, KQA), representing Service Agent duties, limiting insights into other CRM roles (e.g., Service Manager, Service Analyst) or broader enterprise tasks.

3. **Synthetic Dataset Constraints:** The CRMarena dataset [3], a synthetic model of a mid-sized shoe company (schema provided in Appendix A), may not fully represent larger or dynamic CRM systems, potentially limiting the MAS's applicability.
4. **Complex Reasoning Challenges:** The lower PVI accuracy (59.23% for 70B) indicates limitations in tasks requiring intricate reasoning, such as navigating complex object dependencies (e.g., Case to Knowledge__kav, Appendix B, Figure 13: CRMarena Sandbox Schema Diagram).
5. **Inference Cost Scalability:** High inference costs for the 405B model (e.g., \$0.85 for KQA) suggest scalability challenges for resource-constrained environments, with no exploration of optimization techniques (e.g., model quantization).

These limitations suggest that while the MAS achieved high accuracy within the study's scope, its effectiveness in diverse, real-world CRM contexts requires further validation.

Chapter 6

Conclusion

6.1 Alignment with Project Aims and Objectives

The project aimed to develop and evaluate a multi-agent system to automate e-commerce CRM tasks with high accuracy, cost-effectiveness, and semantic reliability, targeting SMEs. The MAS's high NED and KQA accuracies and low inference costs for the 70B model fully align with these goals, demonstrating practical applicability for product identification and customer query resolution. The robust KQA performance, validated by BERTScore, supports the objective of delivering contextually accurate responses, critical for customer satisfaction. The comparison with prior studies confirms the MAS's superior performance, validating the multi-agent approach. However, the lower PVI accuracy and high 405B model costs indicate partial achievement of comprehensive automation and scalability, suggesting that complex reasoning tasks require further refinement. The methodological limitations, including limited model diversity, narrow task scope, and synthetic dataset constraints, highlight areas where the project's scope could be expanded to fully meet the objective of broad CRM applicability.

6.2 Summary of Key Results

The MAS, leveraging task-specific agents, workflow decomposition, and the LLaMA-3.1-70B model with LangGraph, achieved an overall average accuracy of 78.97% across NED, PVI, and KQA tasks. For NED, the 70B model attained a high accuracy of 92.31% (120/130 queries), demonstrating robust performance in identifying products using structured data mappings. KQA performance was also strong, with an accuracy of 85.38% (111/130 queries), supported by BERTScore F1 results showing high semantic accuracy (e.g., Q2: $F1 = 1.0$, Q8: $F1 \approx 0.956$) at a threshold of $F1 \geq 0.85$.

However, PVI accuracy was lower at 59.23% (77/130 queries), indicating challenges in complex policy violation detection. Inference costs for the 70B model were notably low (e.g., \$0.04 for NED, \$0.16 for KQA), contrasting with the higher costs of the 405B model (e.g., \$0.85 for KQA), as shown in Table 3 in Chapter 4. The example predictions in Table 5 in Chapter 4 confirmed accurate outputs for NED (correct Product ID) and KQA ($F1 = 88.07$ for golf shoe features), with PVI correctly identifying a Knowledge Article ID but reflecting lower overall accuracy. These results, visualized in the Django/React + Vite prototype (provided in

Appendix C.3 Prototype UI screenshots), underscore the MAS’s capability for CRM tasks, particularly NED and KQA.

6.3 Conclusions Drawn from Results

The results demonstrate that the MAS effectively automates key CRM tasks, particularly NED and KQA, achieving high accuracies (92.31% and 85.38%, respectively) and reliable KQA responses ($F1 \geq 0.85$), meeting the project’s objective of delivering accurate and contextually relevant solutions for e-commerce CRM. The 70B model’s cost-effectiveness (\$0.04–\$0.16) supports its suitability for SMEs, aligning with the goal of developing a scalable, resource-efficient system. The BERTScore threshold of $F1 \geq 0.85$ ensured semantic accuracy for customer-facing queries, as seen in high-scoring responses, fulfilling the objective of enhanced contextual understanding. However, the lower PVI accuracy (59.23%) highlights limitations in handling complex reasoning tasks, such as navigating intricate object dependencies, which partially meets the objective of comprehensive task automation. Compared to prior studies (Table 6 in Chapter 5), the MAS’s 78.97% accuracy outperforms single-agent systems and competes favourably with Thakkar & Yadav’s multi-agent system [14], reflecting the effectiveness of task-specific agents and workflow decomposition.

6.4 Future Directions

The results and limitations suggest several directions for future research. Incorporating diverse LLMs (e.g., GPT-4) could enhance performance comparisons and address model diversity limitations. Expanding the task scope to include additional CRM roles (e.g., Service Manager) and testing on real-world datasets could improve generalizability. Optimizing PVI reasoning through advanced agent coordination or fine-tuning could address complex reasoning challenges. Additionally, exploring cost-optimization techniques (e.g., model quantization) could enhance scalability for the 405B model, aligning with the goal of cost-effective CRM automation. These improvements would strengthen the MAS’s potential for widespread adoption in e-commerce CRM systems.

Chapter 7

Recommendations

7.1 Enhancing PVI Performance

The MAS achieved a lower accuracy for PVI, indicating challenges in complex reasoning tasks involving intricate object. To improve PVI performance, the following actions are recommended:

- **Develop Advanced Agent Coordination:** Implement specialized PVI agents with enhanced reasoning capabilities, such as chaining multiple reasoning steps using LangGraph, to improve policy violation detection accuracy.
- **Fine-Tune Models for PVI:** Fine-tune LLaMA-3.1 models on a curated subset of CRMArena's Knowledge__kav articles and Case records to better handle policy-specific queries, reducing errors in complex scenarios.
- **Incorporate Rule-Based Heuristics:** Integrate rule-based systems alongside LLMs to enforce strict policy compliance checks, complementing the MAS's data-driven approach.

7.2 Optimizing Inference Cost Scalability

The high inference costs of the 405B model (e.g., \$0.85 for KQA, Table 3 in Chapter 4) highlight scalability challenges for resource-constrained SMEs. To enhance cost-effectiveness, the following actions are recommended:

- **Explore Model Optimization Techniques:** Apply model quantization or pruning to reduce the computational demands of the 405B model, lowering inference costs while maintaining accuracy.
- **Batch Processing for Queries:** Implement batch processing for CRM queries to optimize API calls, reducing costs for high-volume tasks like KQA.
- **Hybrid Model Deployment:** Deploy the 70B model for cost-sensitive tasks (e.g., NED, \$0.04) and reserve the 405B model for high-stakes tasks requiring nuanced responses, balancing cost and performance.

These strategies would make the MAS more scalable, supporting its adoption in SME e-commerce environments.

7.3 Expanding Model and Task Diversity

The limitation of using only LLaMA-3.1 models (70B and 405B) and focusing on three tasks (NED, PVI, KQA) restricts the MAS's generalizability. To address this, the following actions are recommended:

- **Test Diverse LLMs:** Evaluate the MAS with other advanced models (e.g., GPT-4, used in CRMArena [3] and WorkArena [8]) to benchmark performance and identify optimal model configurations for CRM tasks.
- **Include Additional CRM Roles:** Expand the task scope to include other CRMArena roles (e.g., Service Manager, Service Analyst), testing the MAS's versatility across diverse CRM functions.
- **Incorporate Multimodal Inputs:** Integrate multimodal data (e.g., images, customer reviews), as in Thakkar & Yadav [14], to enhance the MAS's ability to handle complex e-commerce queries.

These efforts would broaden the MAS's applicability, addressing the narrow task scope and model diversity limitations.

7.4 Practical Deployment for SMEs

The MAS's high NED and KQA accuracies (92.31% and 85.38%, respectively) and low 70B model costs (\$0.04–\$0.16) make it suitable for SME e-commerce CRM (Section 6.2). To facilitate deployment, the following actions are recommended:

- **Develop User-Friendly Interfaces:** Enhance the Django/React + Vite prototype (Appendix C.3 Prototype UI screenshots, Figures 14 - 19) with intuitive dashboards for SMEs to monitor MAS performance and query outcomes.
- **Provide Training for Stakeholders:** Offer training programs for SME staff to integrate the MAS into existing CRM workflows, leveraging visualizations for decision-making.
- **Pilot Deployment Programs:** Initiate pilot programs with SMEs to test the MAS in operational settings, gathering feedback to refine its functionality and usability.

These actions would support practical adoption, aligning with the project's aim of delivering cost-effective CRM automation for SMEs.

References

- [1] Salesforce, "Global CRM software market share by vendor in 2022," *Statista*, Jun. 2023. Available: <https://www.statista.com/statistics/1234567/crm-software-market-share/>
- [2] A. Payne and P. Frow, "Strategic Customer Management: Integrating Relationship Marketing and CRM." Cambridge, U.K.: Cambridge Univ. Press, 2013.
- [3] KH. Huang, P. Akshara, Sidharth Dhawan, Y. Mao Huan Wang, S. Savarese, C. Xiong, Philippe Laban, Chien-Sheng Wu (2024, Nov). "CRMarena: Understanding the Capacity of LLM Agents to Perform Professional CRM Tasks in Realistic Environments" Salesforce AI Research. Available: <https://arxiv.org/pdf/2411.02305>
- [4] Salesforce. (n.d.). CRM Automation. Retrieved from <https://www.salesforce.com/crm/automation/>
- [5] Medium. (2018). Artificial Intelligence and the Future of CRM. Retrieved from <https://medium.com/the-mission/artificial-intelligence-and-the-future-of-crm-531844d29a1a>
- [6] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2023. "React: Synergizing reasoning and acting in language models." In *The Eleventh International Conference on Learning Representations*. Available: https://openreview.net/forum?id=WE_vluYUL-X
- [7] H. Wang, Z. Liu, Y. Zhang, W. Zhao, Z. Chen, and C. Li. 2024. "Exploration of LLM Multi-Agent Application Implementation Based on LangGraph + CrewAI." Preprint, arXiv:2411.18241. Available: <https://arxiv.org/pdf/2411.18241>
- [8] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. 2024. "Workarena: How capable are web agents at solving common knowledge work tasks?" In *Forty-first International Conference on Machine Learning*. Available: <https://openreview.net/forum?id=BRfqYrikdo>

- [9] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. "Webarena: A realistic web environment for building autonomous agents." In *The Twelfth International Conference on Learning Representations*. Available: <https://openreview.net/forum?id=oKn9c6ytLx>
- [10] Ori Yoran, Samuel Joseph Amouyal, Chaitanya Malaviya, Ben Bogin, Ofir Press, and Jonathan Berant. 2024. "Assistantbench: Can web agents solve realistic and time-consuming tasks?" Preprint, arXiv:2407.15711. Available: <https://arxiv.org/abs/2407.15711>
- [11] Olly Styles, Sam Miller, Patricio Cerda-Mardini, Tanaya Guha, Victor Sanchez, and Bertie Vidgen. 2024. "Workbench: a benchmark dataset for agents in a realistic workplace setting." In *First Conference on Language Modeling*. Available: <https://openreview.net/forum?id=4HNAwZFDcH#discussion>
- [12] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yu, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. "SWE-bench: Can language models resolve real-world github issues?" In *The Twelfth International Conference on Learning Representations*. Available: <https://openreview.net/forum?id=VTF8yNQM66>
- [13] Wang, Z., et al. 2024b. "Multi-agent collaboration framework for recommender systems." Available: <https://arxiv.org/pdf/2402.15235>
- [14] Thakkar, P., & Yadav, A. 2024. "Personalized Recommendation Systems using Multimodal, Autonomous, Multi-Agent Systems." Available: <https://arxiv.org/abs/2410.19855>

Appendices

Appendix A

CRMArena Schema (json format)

```
[
  {
    "object": "User",
    "fields": {
      "Alias": "Alias, usually a shortened version of the agent's name",
      "Email": "Email address of the agent",
      "EmailEncodingKey": "Email encoding, usually 'UTF-8'",
      "FirstName": "First name of the agent",
      "LanguageLocaleKey": "Language locale, e.g., 'en_US'",
      "LastName": "Last name of the agent",
      "LocaleSidKey": "Locale, e.g., 'en_US'",
      "Phone": "Phone number of the agent",
      "ProfileId": "ID of the user profile, typically 'Standard User' or a similar profile",
      "TimeZoneSidKey": "Time zone, e.g., 'America/Los_Angeles'",
      "Username": "Unique username for the agent in Salesforce"
    }
  },
  {
    "object": "Account",
    "fields": {
      "FirstName": "First name of the customer",
      "LastName": "Last name of the customer",
      "PersonEmail": "Email address of the customer",
      "Phone": "Phone number of the customer",
      "RecordTypeId": "Record type ID, typically for 'Person Account'",
      "ShippingCity": "Shipping city of the customer",
      "ShippingState": "Shipping state of the customer"
    }
  },
  {
    "object": "Contact",
    "fields": {
      "AccountId": "References Account object",
      "Email": "Email address of the contact",
      "FirstName": "First name of the contact",
      "LastName": "Last name of the contact",
      "OwnerId": "References User object"
    }
  },
  {
    "object": "Case",
```

```

    "fields": {
      "AccountId": "References Account object",
      "Channel": "Channel through which the case was initiated",
      "ClosedDate": "Date when the case was closed in 'YYYY-MM-DDTHH:MM:SSZ'
format",
      "ContactId": "ID of the contact associated with this case. References
Contact object",
      "CreatedDate": "Date when the case was created in 'YYYY-MM-DDTHH:MM:SSZ'
format",
      "Description": "Detailed description of the case",
      "IssueId__c": "ID of the Issue associated with this case",
      "OrderItemId__c": "ID of the Order Item associated with this case",
      "OwnerId": "The latest owner of the case. References User object",
      "Priority": "Priority of the case",
      "Status": "Status of the case, e.g., 'New', 'Working', 'Escalated',
etc",
      "Subject": "Subject of the case"
    }
  },
  {
    "object": "Knowledge__kav",
    "fields": {
      "FAQ_Answer__c": "Content of the knowledge article",
      "Summary": "Summary of the article",
      "Title": "Title of the knowledge article",
      "UrlName": "Unique URL for the article"
    }
  },
  {
    "object": "ProductCategory",
    "fields": {
      "CatalogId": "ID of the catalog this category belongs to",
      "Name": "Name of the product category"
    }
  },
  {
    "object": "Product2",
    "fields": {
      "Description": "Description of the product",
      "External_ID__c": "External ID for mapping purposes",
      "IsActive": "Boolean to mark if the product is active",
      "Name": "Name of the product"
    }
  },
  {
    "object": "ProductCategoryProduct",
    "fields": {
      "Product": "External ID of the product",

```



```

        "ProductCategory": "Name of the product category",
        "ProductCategoryId": "References ProductCategory object",
        "ProductId": "References Product2 object"
    },
    {
        "object": "Pricebook2",
        "fields": {
            "Description": "Description of the price book",
            "IsActive": "Boolean to mark if the price book is active",
            "Name": "Name of the price book",
            "ValidFrom": "Start date for the price book in 'YYYY-MM-DDTHH:MM:SSZ'
format",
            "ValidTo": "End date for the price book in 'YYYY-MM-DDTHH:MM:SSZ'
format"
        }
    },
    {
        "object": "PricebookEntry",
        "fields": {
            "Pricebook2Id": "ID of the price book this entry belongs to. References
Pricebook2 object",
            "Product2Id": "ID of the product associated with this price book entry.
References Product2 object",
            "UnitPrice": "Unit price of the product"
        }
    },
    {
        "object": "Order",
        "fields": {
            "AccountId": "ID of the customer account. References Account object",
            "EffectiveDate": "Effective date of the order in 'YYYY-MM-DD' format",
            "Pricebook2Id": "ID of the price book this order belongs to. References
Pricebook2 object",
            "Status": "Status of the order, typically 'Draft' or 'Activated'"
        }
    },
    {
        "object": "OrderItem",
        "fields": {
            "OrderId": "ID of the order this item belongs to. References Order
object",
            "PriceBookEntryId": "ID of the price book entry for this order item",
            "PricebookEntryId": "References PricebookEntry object",
            "Product2Id": "ID of the product associated with this order item.
References Product2 object",
            "Quantity": "Quantity of the product ordered",
            "UnitPrice": "Unit price of the product"
        }
    }
}

```

```

    }
  },
  {
    "object": "EmailMessage",
    "fields": {
      "FromAddress": "Sender email address",
      "MessageDate": "Timestamp when the message was sent",
      "ParentId": "ID of the case related to this email. References Case
object",
      "Subject": "Subject of the email",
      "TextBody": "Content of the email",
      "ToIds": "List of recipient IDs"
    }
  },
  {
    "object": "LiveChatTranscript",
    "fields": {
      "AccountId": "ID of the customer account. References Account object",
      "Body": "Content of the chat transcript",
      "CaseId": "ID of the case related to this chat. References Case object",
      "ContactId": "References Contact object",
      "EndTime": "Timestamp when the chat ended",
      "LiveChatVisitorId": "ID of the chat visitor",
      "OwnerId": "ID of the agent who owned the chat. References User object"
    }
  },
  {
    "object": "Issue__c",
    "fields": {
      "Description__c": "Detailed description of the issue",
      "Name": "The name/title of the issue"
    }
  },
  {
    "object": "CaseHistory__c",
    "fields": {
      "CaseId__c": "ID of the case this history entry is related to.
References Case object",
      "CreatedDate": "The date when this change was made",
      "Field__c": "Specifies the field or type of change. One of ['Case
Creation', 'Owner Assignment', 'Case Closed']",
      "NewValue__c": "The new value after the change. References User object
if the type of change is 'Owner Assignment' else null",
      "OldValue__c": "The previous value before the change. References User
object if the type of change is 'Owner Assignment' else null"
    }
  }
]

```

Appendix B

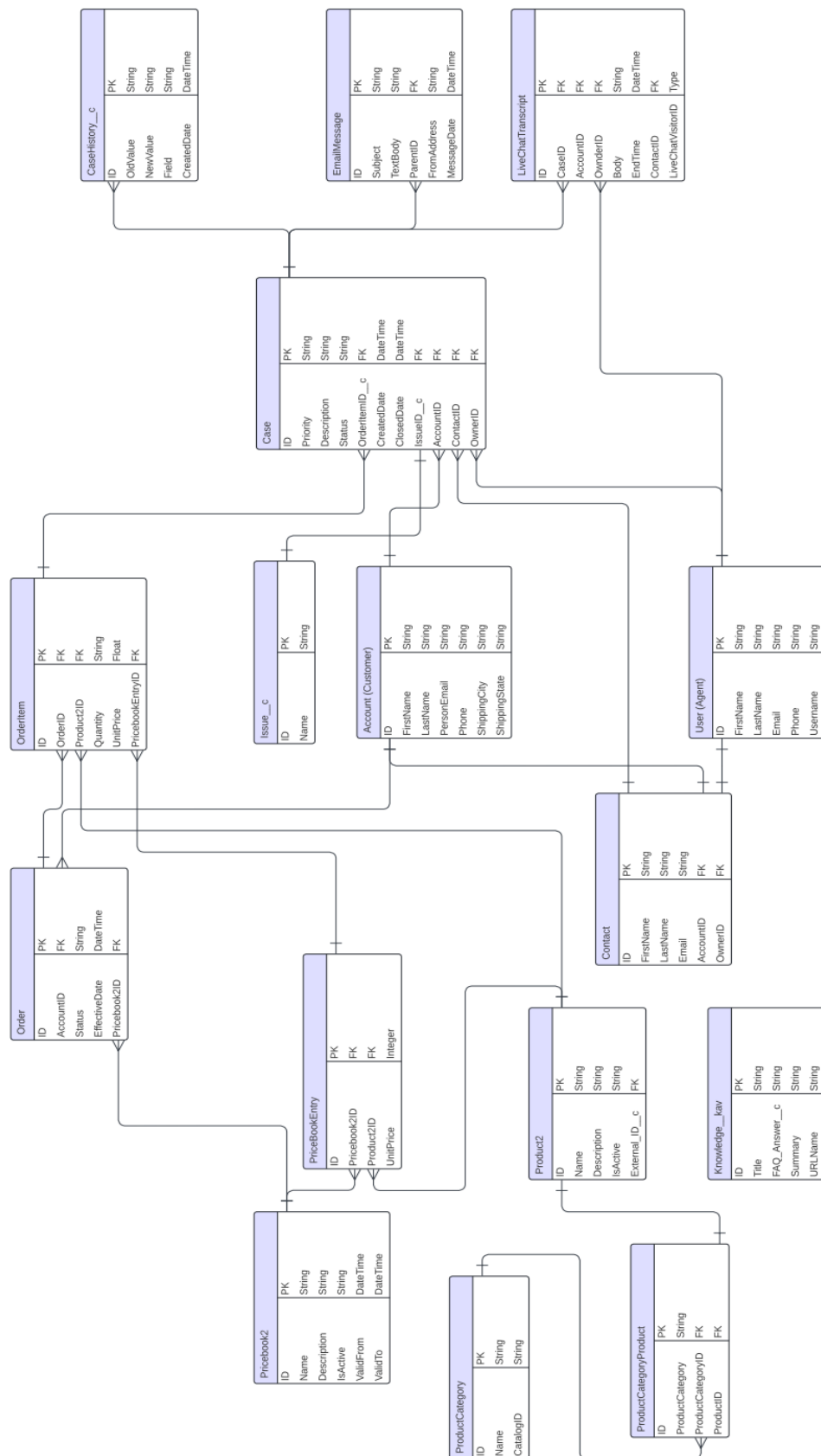


Figure 13: CRM Arena Sandbox Schema Diagram

Appendix C

Implementation Details

C.1 Prompts

Below are key prompts and queries used in the system implementation, as referenced in the Implementation section:

C.1.1 Task Classifier Prompt

```
classification_prompt = f"""
```

```
You are a Salesforce Task Classifier Agent in a CRM automation system.  
Your job is to classify user queries into one of the following three  
categories based on their intent and wording:
```

```
1. NED (Named Entity Disambiguation)
```

```
These queries refer to specific products that the customer has already  
purchased.
```

```
They often include time filters or product descriptions.
```

```
Examples:
```

- "Display the waterproof hiking boots I purchased three days back"
- "Can you display the adjustable weight dumbbells I purchased a month ago?"
- "Can you display the women's trail jacket I purchased a fortnight ago?"

```
2. PVI (Policy Violation Identification)
```

```
These queries question whether an agent followed company policy during a  
specific interaction or case.
```

```
Examples:
```

- "Did the agent breach the policy in this situation?"
- "Did the agent breach the policy in this instance?"
- "Did the agent breach the policy?"

```
3. KQA (Knowledge Question Answering)
```

```
These are general knowledge-based questions not tied to a specific purchase  
or agent behavior.
```

```
They often relate to product features, company policies, or usage  
instructions.
```

```
Examples:
```

- "What technology features do Shoes & Clothings golf shoes have?"
- "What is the time frame for being eligible for a full refund with Shoes & Clothings?"
- "How can I store my yoga straps to prolong their life?"

```
Now classify the following user query into only one of the above  
categories.
```

```
User Query: "{query}"
```

Respond with exactly one word: NED, PVI, or KQA.
"""

C.1.2 NED Query Parsing Prompt

```
template="""
You are a helpful assistant tasked with extracting information from
a user's query about a past purchase. The query is: "{query_text}"

Extract the following:
1. The name of the product purchased.
2. The number of days before today that the purchase was made.

{format_instructions}

Examples:
- Query: "Can you display the women's trail jacket I purchased a
fortnight ago?"
Output: {"product_name": "women's trail jacket", "days": 14}}
- Query: "Can you display the waterproof swim cap I purchased three
days back?"
Output: {"product_name": "waterproof swim cap", "days": 3}}
- Query: "Can you display the high-tech fitness bra I purchased last
week?"
Output: {"product_name": "high-tech fitness bra", "days": 7}}

Query: "{query_text}"
```

C.1.3 PVI LLM Prompt (Violation Check)

```
"""You are a Salesforce support analyst. Your task is to analyze a
case description and match it to the most relevant knowledge article
summary, based on the issue described.

Instructions:
- Match the case to a knowledge article only if the summary
addresses the same kind of issue (policy, process, or escalation).
- For product issues (e.g., size mismatches, incorrect items), match
to an article if it explicitly addresses solutions like exchanges,
refunds, or replacements.
- If no article covers the issue or the case is vague/general,
return "None".
- If multiple articles seem relevant, pick the most specific one.

Case description:
{case_description}
```

```
Knowledge article summaries:
{article_summaries}
```

Examples:

1. Case: "I am trying to cancel my order for the Altitude Hiking Pants but the website will not allow me to do so despite being within the cancellation window."

Output: "None" (no article covers this issue)

2. Case: "The size chart provided on the website does not accurately represent the actual product size."

Output: "ka0Ws000000QriDIAS"

matched_article_summary: "Shoes & Clothings is dedicated to delivering top-quality athletic footwear, apparel, and equipment, but occasionally, customers may experience size mismatches between the size chart and the received product. This article explores feasible solutions such as product replacement, store credit, and exclusive discount codes to address and resolve these sizing issues effectively."

3. Case: "This article discusses Shoes & Clothings' exchange policy, providing customers with clear guidelines on how to swap their purchased items for different sizes, colors, or models. It includes step-by-step instructions and conditions, ensuring an effortless exchange process."

Output: "ka0Ws000000QrofIAC"

matched_article_summary: "Shoes & Clothings prides itself on providing top-notch products to athletes around the globe, but occasionally, issues like receiving an incorrect item may occur. This article outlines feasible solutions to resolve the problem, including options for a full refund or replacement. Learn how to quickly and efficiently rectify the situation, ensuring a seamless shopping experience and continued satisfaction with our brand."

Return only the article ID (like "ka0Ws000000QsZRIA0") or "None".
"""

C.1.4 KQA Prompt (Answer Extraction)

```
template="""
```

```
You are a Salesforce Knowledge Specialist tasked with answering a query based on Salesforce Knowledge articles. Your role is to extract a precise and concise answer directly from the FAQ_Answer__c field of the most relevant article. The query is: "{query}"
```

```
Below are the relevant Knowledge articles retrieved: {articles}
```

```
Instructions:
```

- Extract the answer directly from the FAQ_Answer__c field of the article that best matches the query.
- If the FAQ_Answer__c field is unavailable or does not fully address the query, provide a concise summary of the most relevant information from the article's content, avoiding extraneous details.
- If multiple articles are relevant, select the one most closely aligned with the query's focus.
- Return "No relevant information found" if no article addresses the query.
- Ensure the answer is brief, exact, and matches the query's intent without adding unnecessary details.
- Format the response as a JSON object with an "answer" key.

```
{format_instructions}
```

Examples:

Query: "What technology features do Shoes & Clothings golf shoes have?"

Answer: "Advanced sole technology and waterproof materials"

Query: "If opting for store credit, within how many days from the purchase date must you request it?"

Answer: "90 days"

Query: "Can I get a full refund for damaged running shoes after 30 days?"

Answer: "No, the request must be submitted within 30 days of the purchase date."

Query: "{query}"

""

C.2 SOQL / SOSL Queries

C.2.1 NED SOQL Query for Account Retrieval

```
SELECT AccountId FROM Contact WHERE Id = '{contact_id}'
```

C.2.2 NED SOQL Query for OrderItem Retrieval

```
SELECT Product2Id, Product2.Name
FROM OrderItem
WHERE Order.AccountId = '{account_id}'
AND Order.EffectiveDate >= {effective_date}
```

C.2.3 PVI SOQL Query for Case Retrieval

```
SELECT Description, IssueId__c, Subject FROM Case WHERE Id = '{case_id}'
```

C.2.4 PVI SOSL Query for Filter relevant article

```
FIND {case_subject_terms} IN ALL FIELDS RETURNING Knowledge__kav(Id, Title, Summary)
```

C.2.5 KQA SOSL Query

```
FIND {golf shoes OR technology} IN ALL FIELDS  
RETURNING Knowledge__kav(Id, Title, FAQ_Answer__c  
WHERE PublishStatus='Online' AND Language='en_US')
```

C.3 Prototype UI screenshots

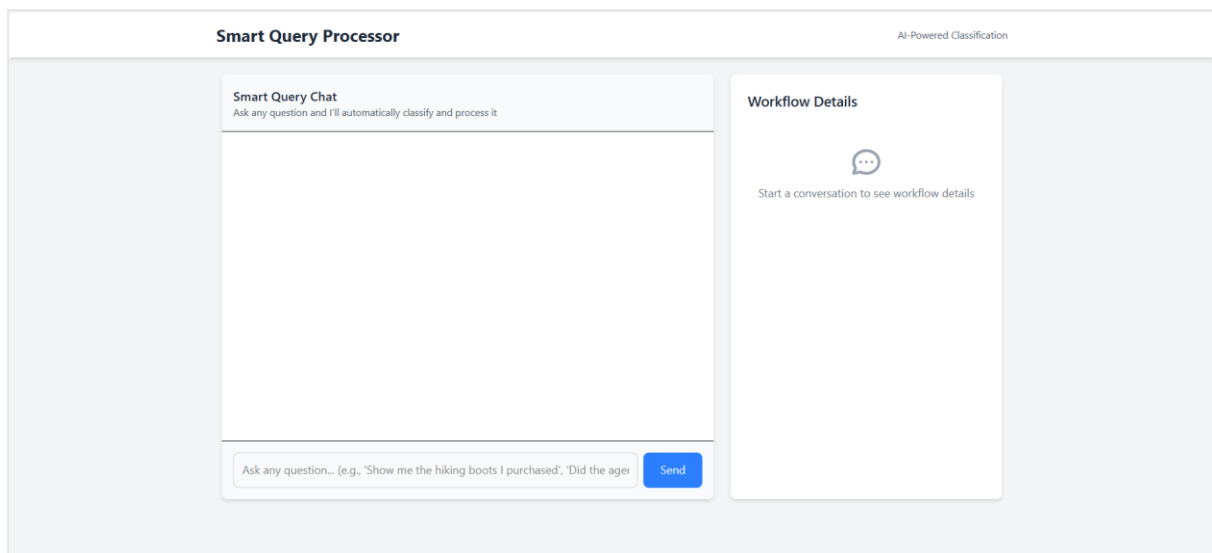


Figure 14: Home Page of Prototype

Smart Query Processor

AI-Powered Classification NED

← Product Information Required

Your query: "Can you display the women's trail jacket I purchased a fortnight ago?"

Please provide the required information to find your purchased products.

Contact ID

003Ws000004Fo3qlAC

Today's Date

06/15/2020

Process Query

Figure 15: NED Task UI

Smart Query Processor

AI-Powered Classification

Smart Query Chat

Ask any question and I'll automatically classify and process it

Can you show me the basketball performance hoodie I purchased two weeks ago?

Classified as: NED

NED Response:

Product: Performance Basketball Hoodie

Product ID: 01tWs000002wSyrIAE

Ask any question... (e.g., 'Show me the hiking boots I purchased', 'Did the age

Send

Workflow Details

NED Workflow

Agent Outputs:

Node 1

Agent 1

product_id: 01tWs000002wSyrIAE

product_name: Performance Basketball Hoodie

match_type: partial

reason: Partial match with common words: {'basketball', 'hoodie', 'performance'}

Final Results:

Product Name: basketball performance hoodie

Product ID: 01tWs000002wSyrIAE

Match Type: partial

Raw Response (Debug)

```
{
  "query_text": "Can you show me the basketball",
  "contact_id": "003Ws000004Fv1G1AS",
  "product_name": "basketball performance hoodi",
  "today_date": "2021-12-13",
  "effective_date": "2021-11-29",
  "account_id": "001Ws00003Ljof0IAB",
  "order_items": [

```

Figure 16:NED Output Page

65

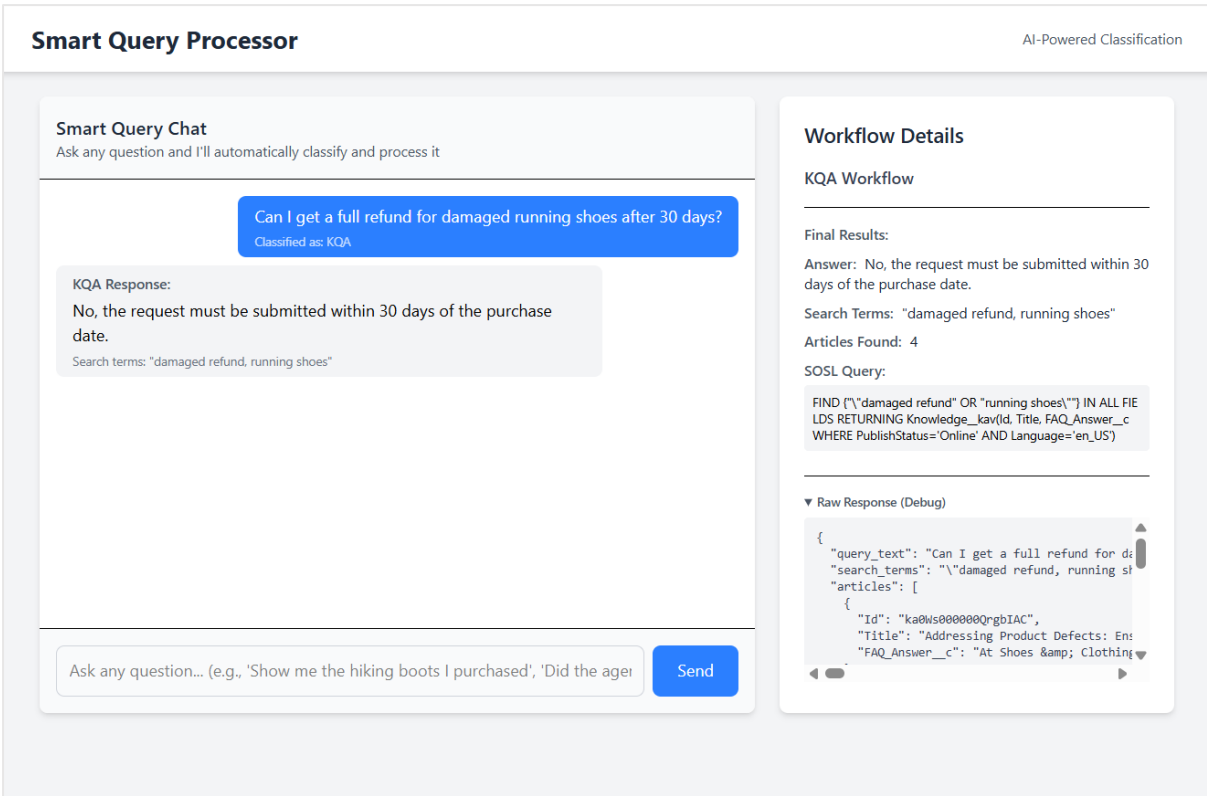


Figure 17: KQA Output Page

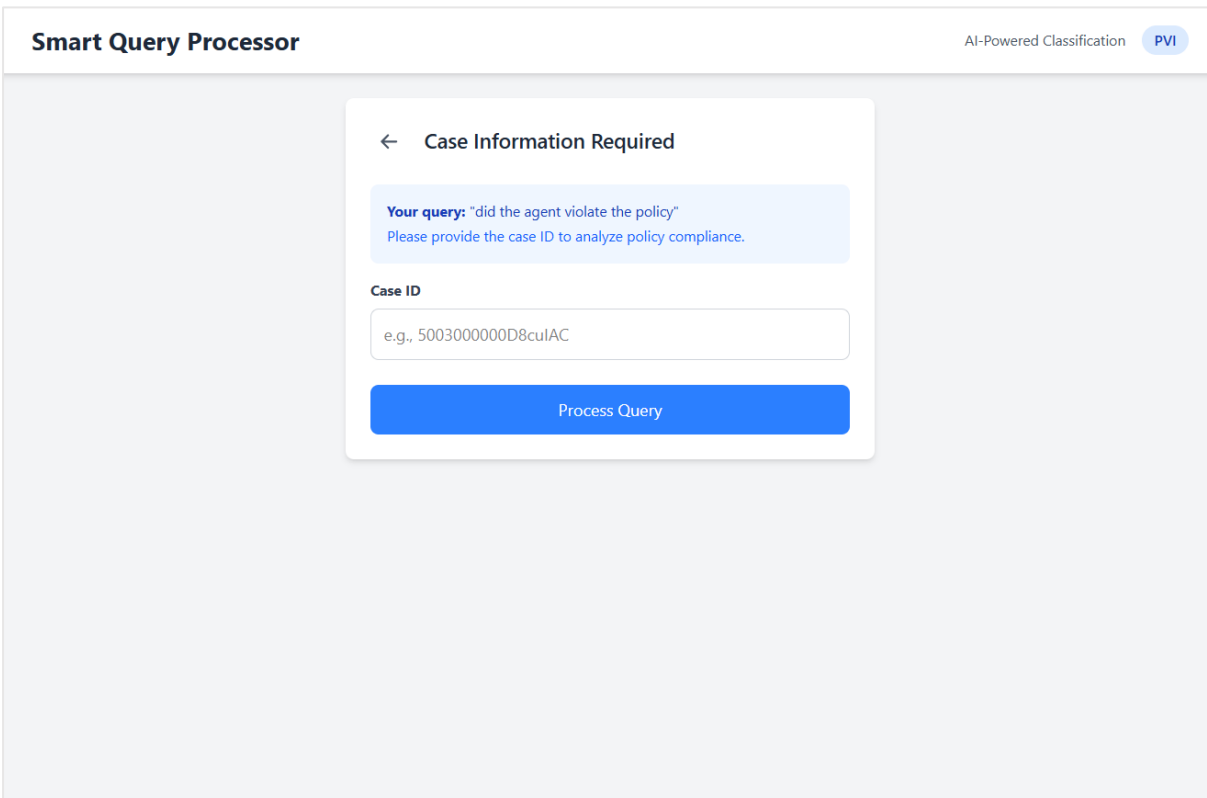


Figure 18: PVI Task UI

Smart Query Processor

AI-Powered Classification

Smart Query Chat

Ask any question and I'll automatically classify and process it

did the agent violates this policy

Classified as: PVI

PVI Response:

Case Description: I received a different item instead of the Retro Style Sneakers I had ordered and I am requesting an immediate exchange.

Case Subject: Received Incorrect Item

Article ID: ka0Ws000000QroflAC

Article Title: How to Address Incorrect Items Received at Shoes & Clothings

Ask any question... (e.g., 'Show me the hiking boots I purchased', 'Did the age

Send

Workflow Details

PVI Workflow

Final Results:

Case ID: 500Ws000007qqKoIAI

Case Description: I received a different item instead of the Retro Style Sneakers I had ordered and I am requesting an immediate exchange.

Case Subject: Received Incorrect Item

Article ID: ka0Ws000000QroflAC

Article Title: How to Address Incorrect Items Received at Shoes & Clothings

▼ Raw Response (Debug)

```
{
  "query_text": "",
  "case_id": "500Ws000007qqKoIAI",
  "case_description": "I received a different :
  "case_subject": "Received Incorrect Item",
  "case_subject_terms": [
    "Received",
    "Incorrect",
```

Figure 19: PVI Output Page